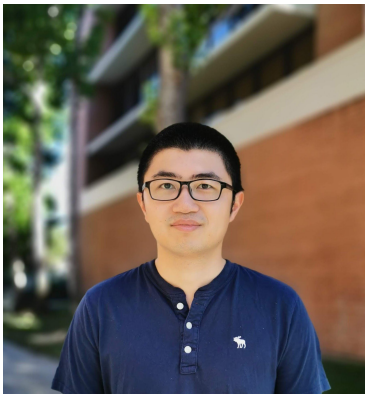


Modern Database Systems

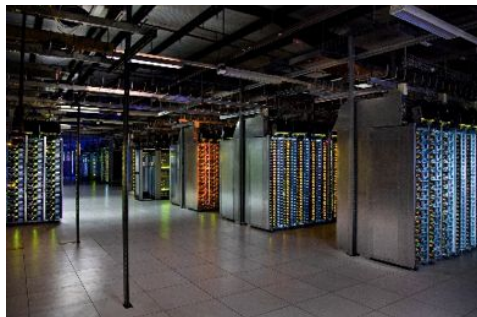
Haoyu Huang

Bio

- Software Engineer at Databricks.
 - Lead compute efforts in Lakebase.
- Ph.D. at USC with Prof. Shahram. 2020
 - Disaggregated database systems.



Today's Data Stores



Data Centers

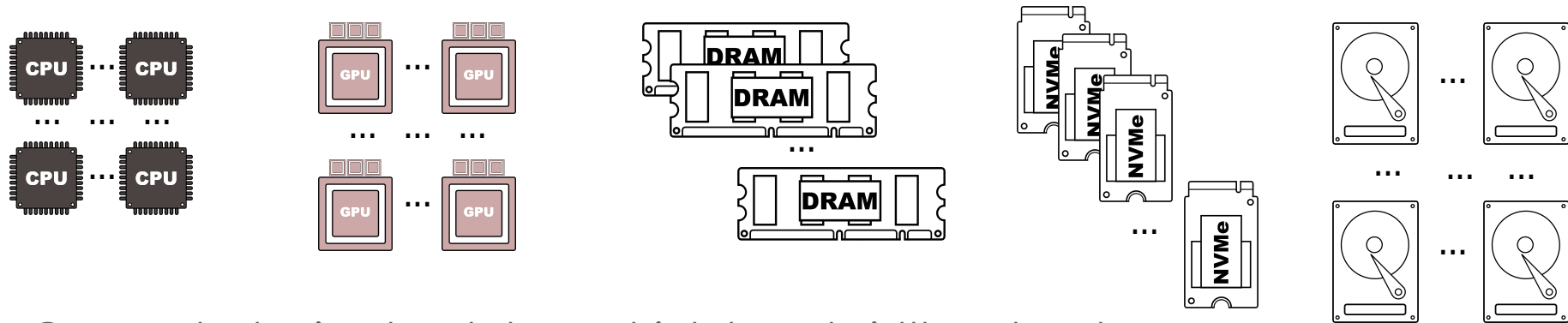


Demanding applications.

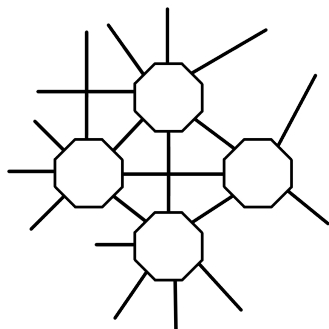
- High Volume: 2.4 billion user profiles, 250 billion friend connections, 3.13 trillion likes, 37 billion tagged locations, 240 billion photos,
- High Velocity: 1.6 billion active users daily, 5.5 billion likes daily, 450 million photos uploaded daily, ...
- High Variety: Mix of data types: Structured records, multimedia content, text.

Modern Data Centers

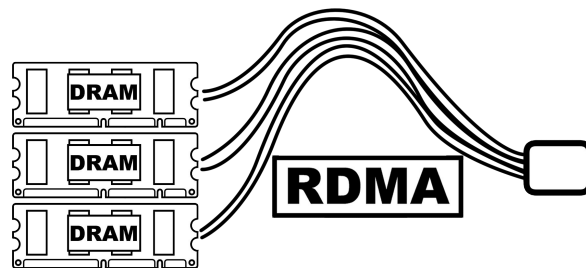
Data centers with disaggregated hardware:



Connected using low latency high bandwidth networks:



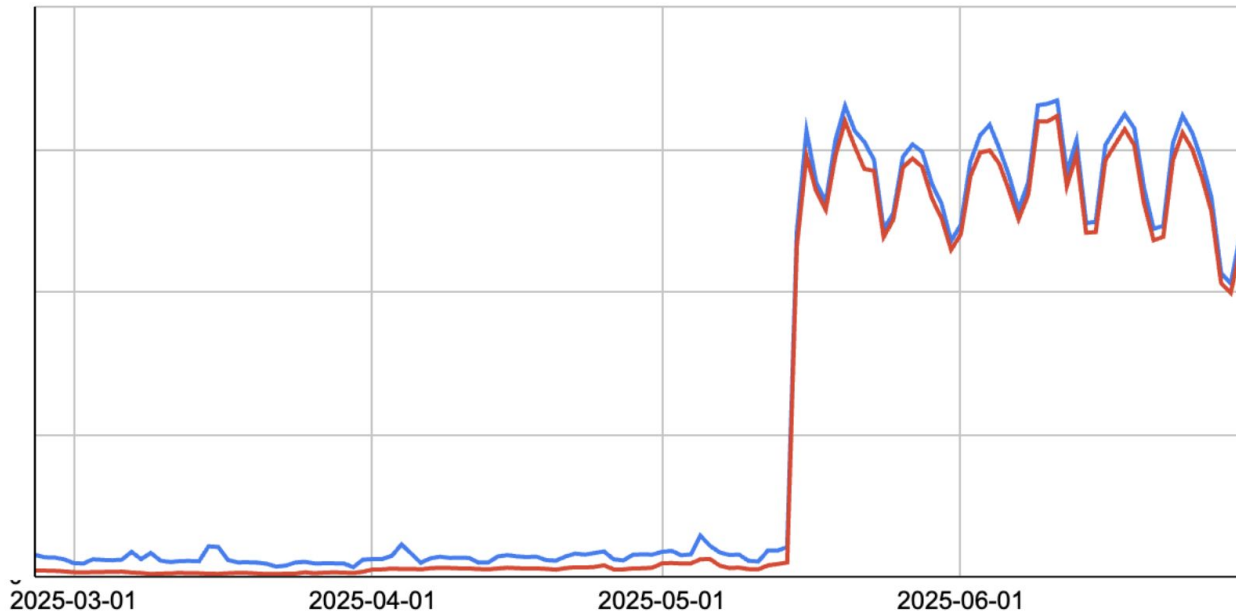
Fast Ethernet



Lakebase

Agent creates many more databases than human.

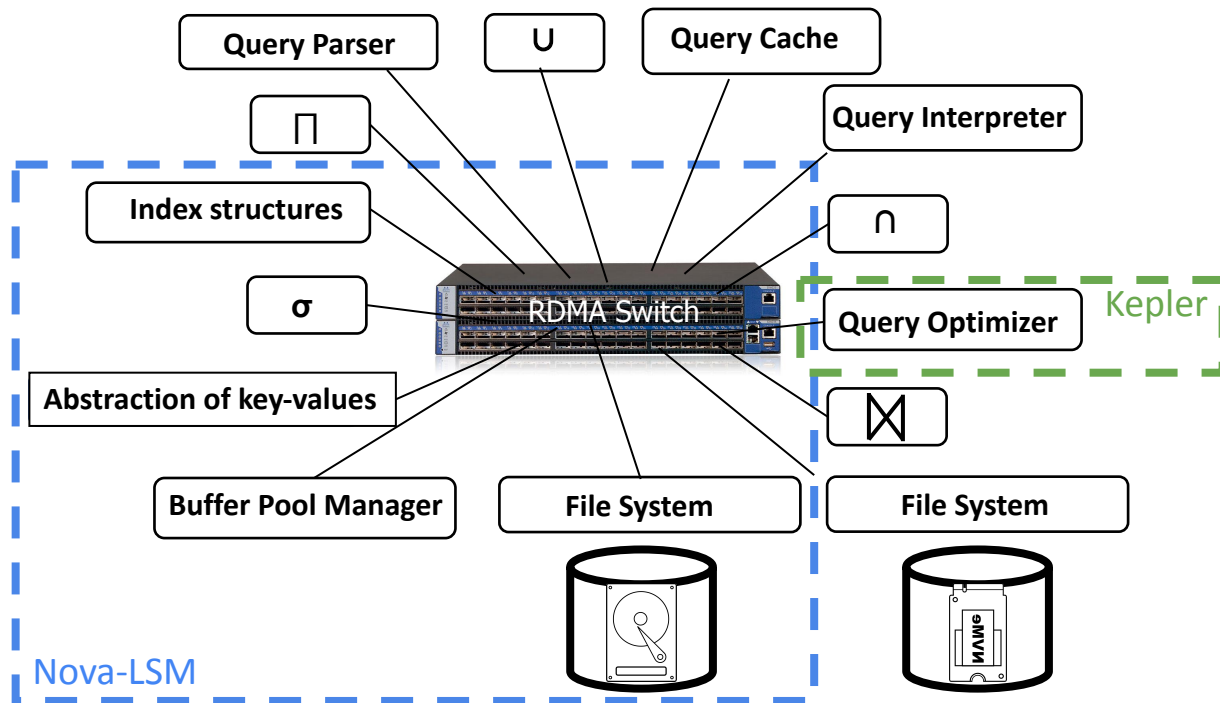
— Total new DB count — Agent created db count



RDMA

- Remote direct memory access (RDMA).
 - > 100 Gbps bandwidth.
 - Single-digit microsecond latency.
 - Communications bypass kernel.
 - One-sided READ/WRITE verbs bypass remote machine's CPU.
 - One machine accesses a remote machine's memory directly.
 - Servers use queue pairs to connect with each other.
 - Two-sided SEND/RECEIVE verbs.

Composable Data Store



- [1] Haoyu Huang and Shahram Ghandeharizadeh. 2021. Nova-LSM: A Distributed, Component-based LSM-tree Key-value Store. In Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21). Association for Computing Machinery, New York, NY, USA, 749–763. <https://doi.org/10.1145/3448016.3457297>
- [2] Lyric Doshi, Vincent Zhuang, Gaurav Jain, Ryan Marcus, Haoyu Huang, Deniz Altinbeken, Eugene Brevdo, and Campbell Fraser. 2023. Kepler: Robust Learning for Parametric Query Optimization. Proc. ACM Manag. Data 1, 1, Article 109 (May 2023), 25 pages. <https://doi.org/10.1145/3588963>
- [3] Ghandeharizadeh, S., Bernstein, P.A., Borthakur, D., Huang, H., Menon, J., Puri, S. (2023). Disaggregated Database Management Systems. In: Nambiar, R., Poess, M. (eds) Performance Evaluation and Benchmarking. TPCTC 2022. Lecture Notes in Computer Science, vol 13860. Springer, Cham. https://doi.org/10.1007/978-3-031-29576-8_3
- [4] Shahram Ghandeharizadeh, Haoyu Huang, and Hieu Nguyen. Nova: Diffused Database Processing using Clouds of Components [Vision Paper]. In 15th IEEE International Conference on Beyond Database Architectures and Structures (BDAS), Ustron, Poland, 2019.

Outline

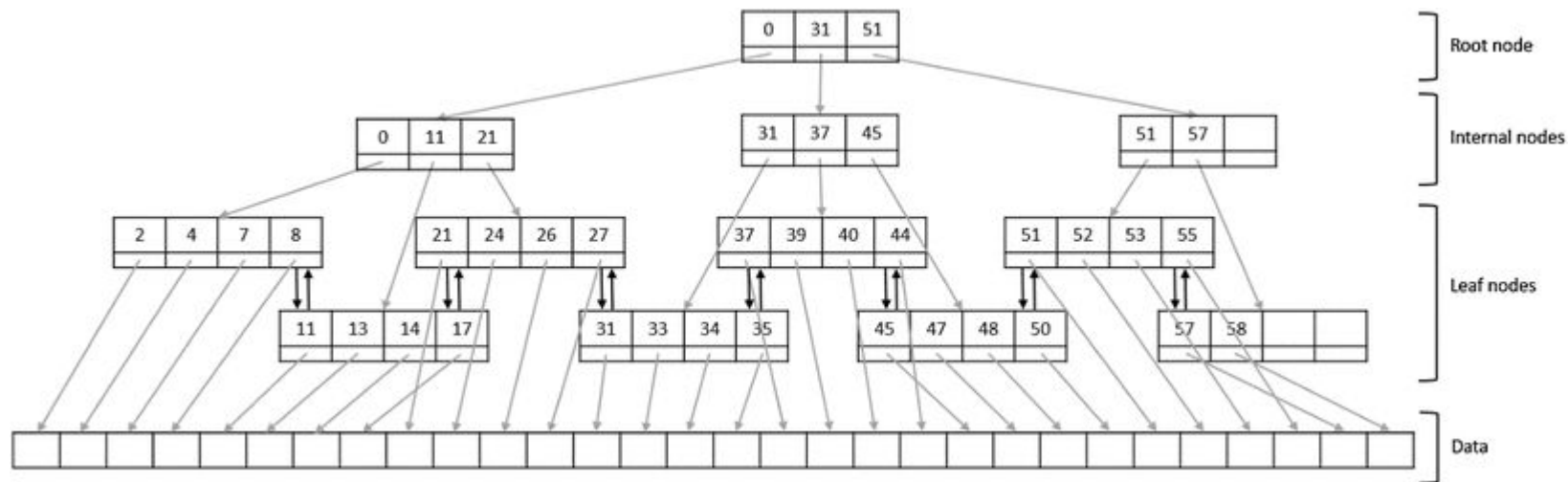
- Part-I:
 - Nova-LSM. A disaggregated LSM-tree. SIGMOD '21
- Part-II:
 - Lakebase. OLTP database built in the AI era.

Key-Value Interfaces

- `Get(key) -> Value`
- `Put(key, value) -> void`
- `Iterate(start key) -> iterator`

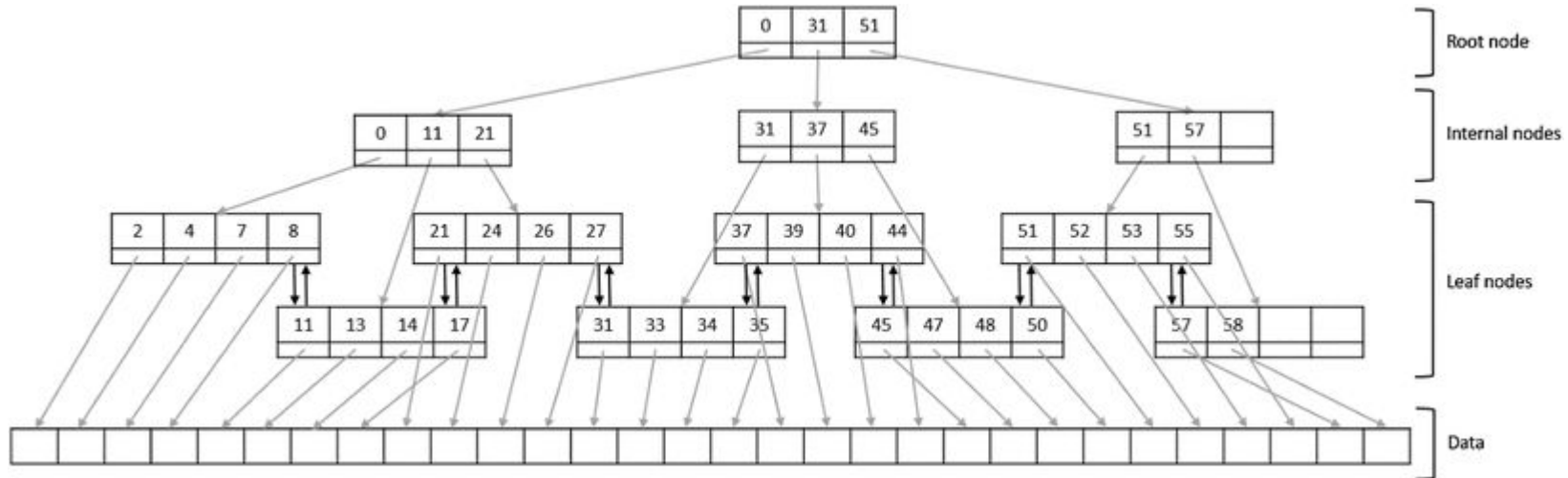
B-trees

- Get. $\log(N)$
- Iterate. $\log(N)$ to locate the starting key.



B-trees

- Put. Expensive. (In-place update)
 - Locate the left page for the key.
 - If the leaf page is full, split the page and continue to cascade split the page till the root.
 - This may end up with many disk seeks. (one seek take multiple ms).



LSM-tree: Log-structured Merge-tree

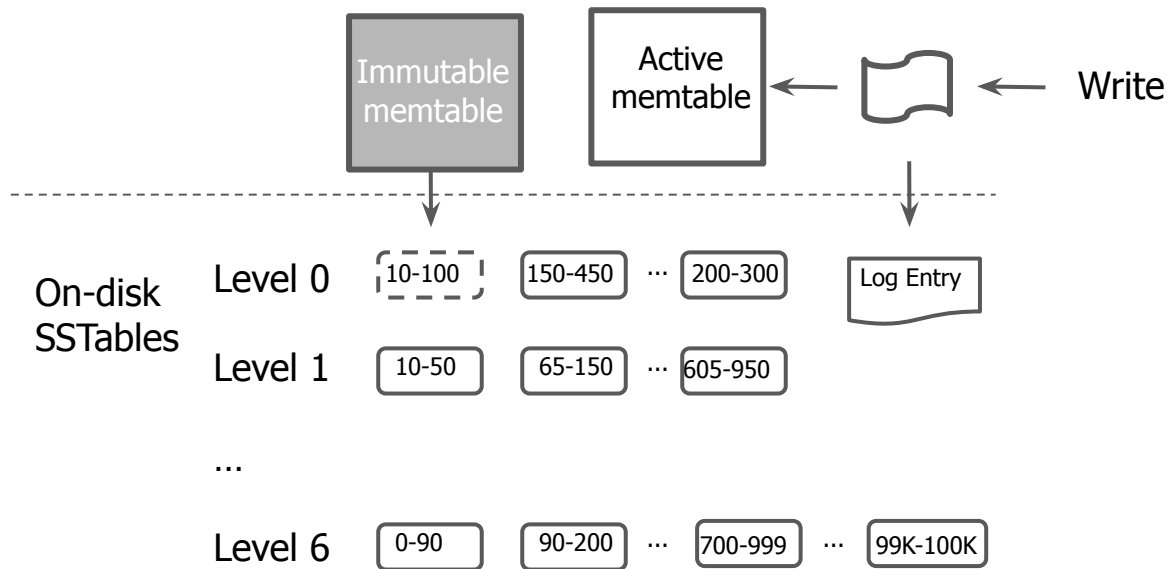
- Introduced in 1996 and implemented in many modern database storage engines.
- LSM-tree optimizes write performance.



LSM-tree and LevelDB

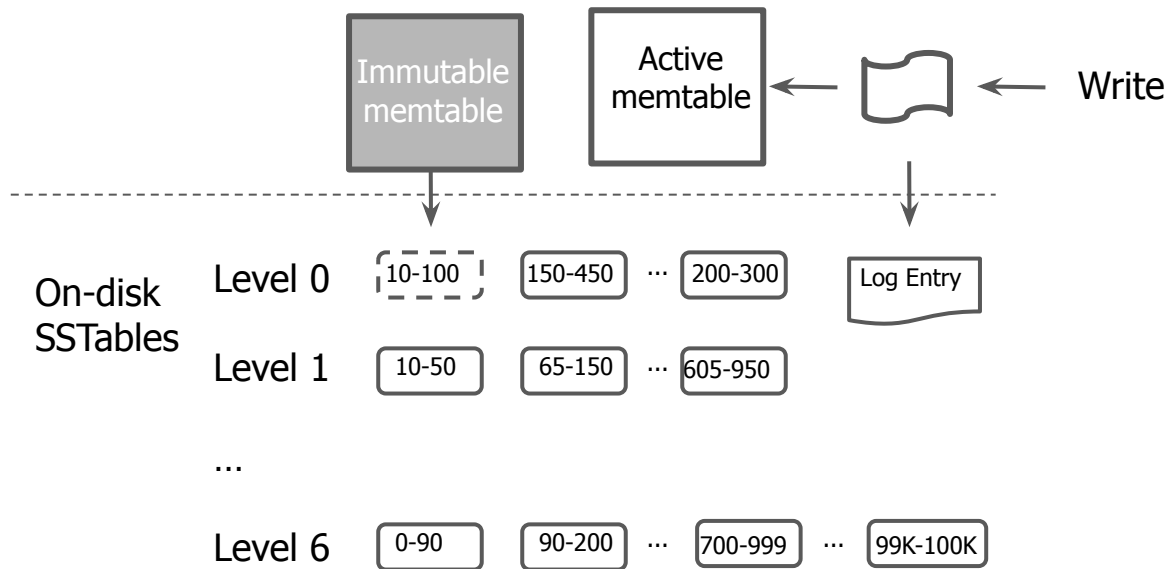
- LSM-tree

- Suitable for write-heavy workload.
- LevelDB: Google's implementation of LSM-tree.



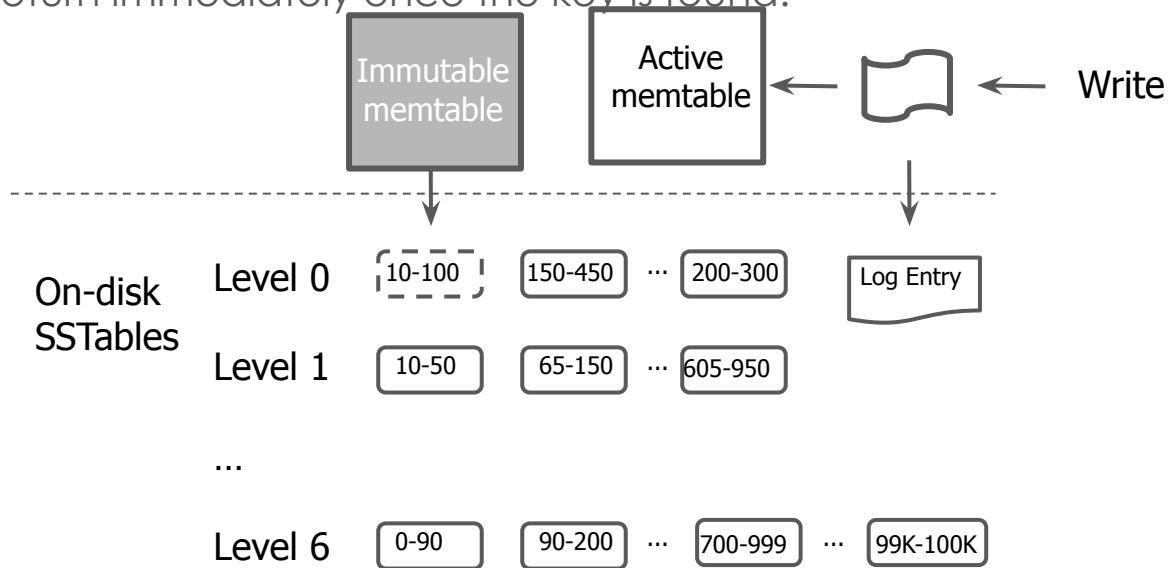
LSM-tree and LevelDB

- Put (key, value):
 - Append to the active memtable. (Skiplist)
 - Append a log record to the log file.
 - Done.



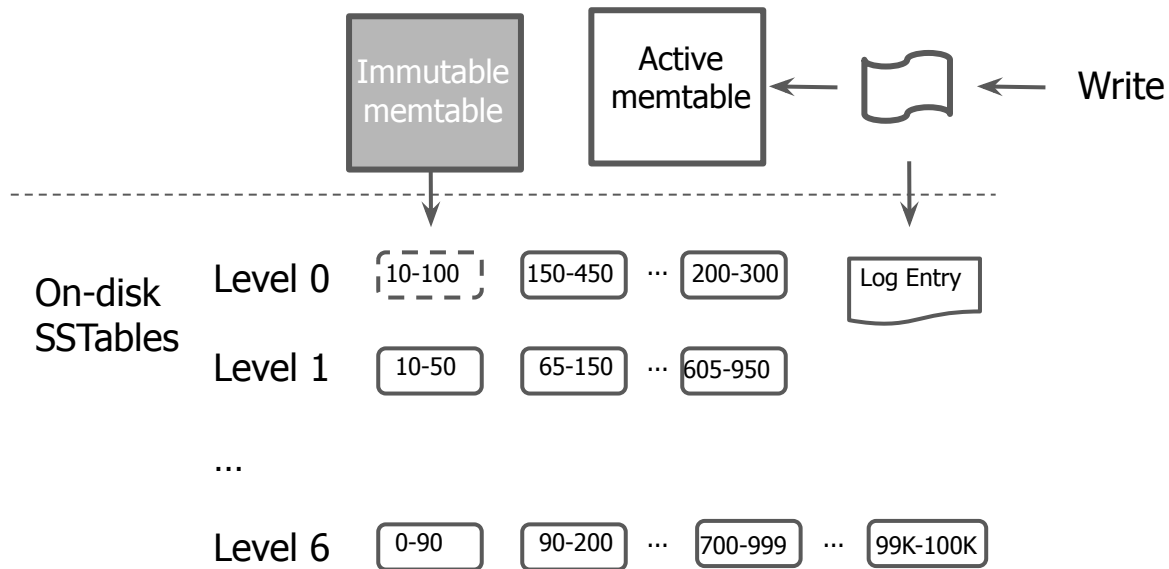
LSM-tree and LevelDB

- Get (key):
 - Search memtables.
 - Search all tables at Level 0.
 - Search the rest SSTables level by level.
 - Return immediately once the key is found.

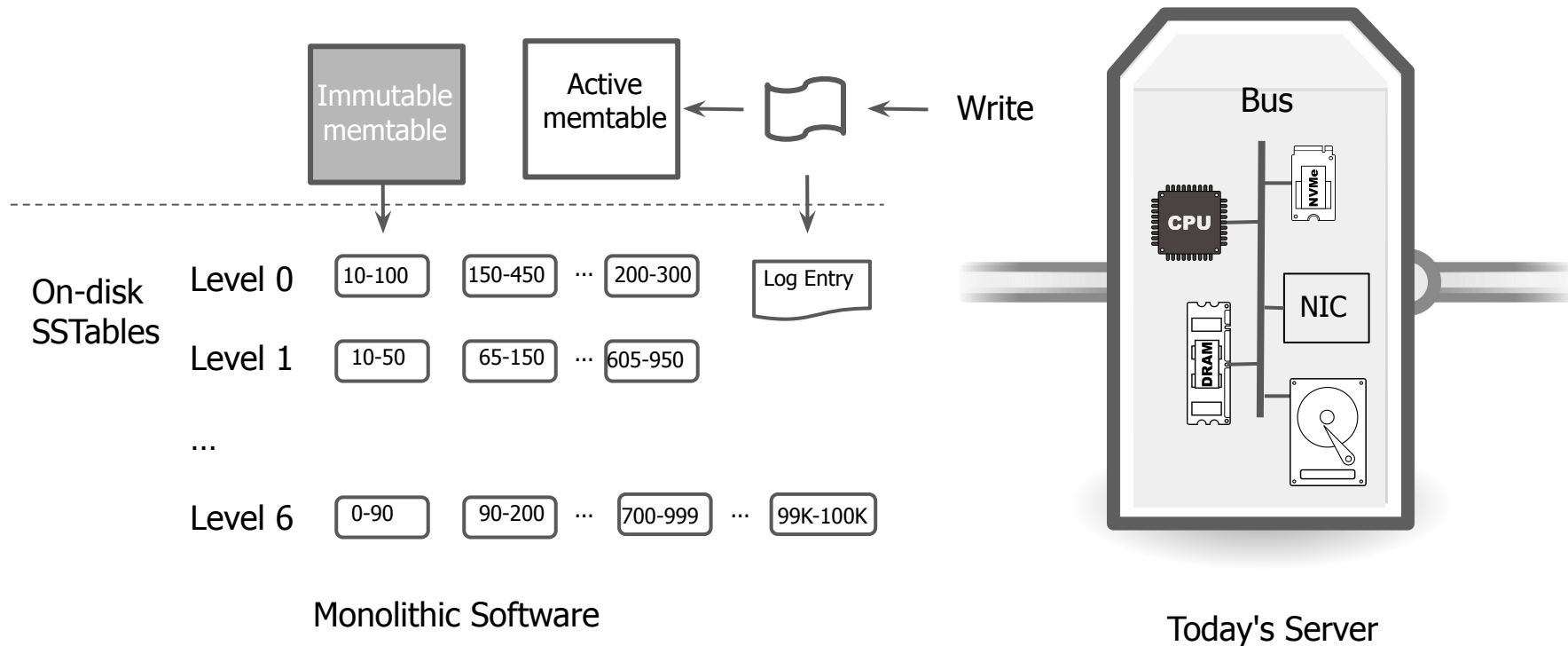


LSM-tree and LevelDB

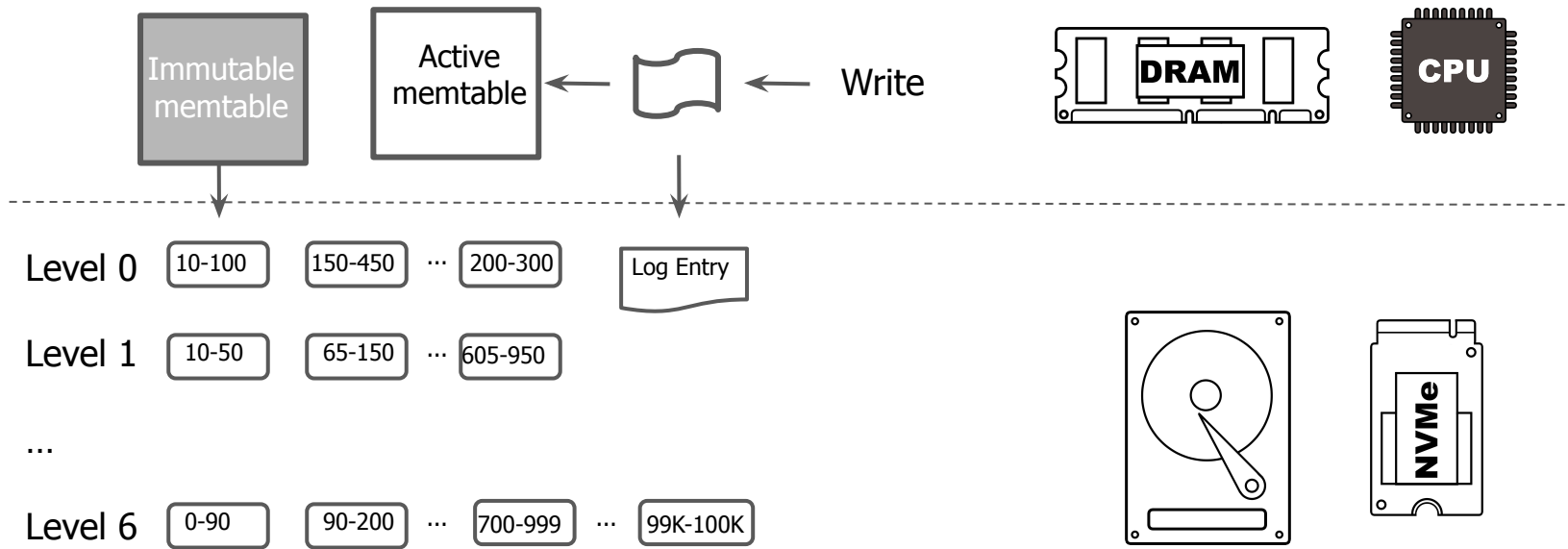
- Iterate (start key):
 - Find the start key similar to get.
 - Then doing k-way merge.
 - $K = \text{memtables} + \text{all tables at Level 0} + \text{one table at each level for level} > 0.$



Today's Key-value Stores, e.g., LevelDB, RocksDB



Nova-LSM: Preliminaries

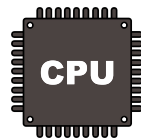


Component-based Software

Nova-LSM: Components

1. LSM-tree Component, LTC:

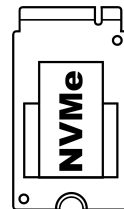
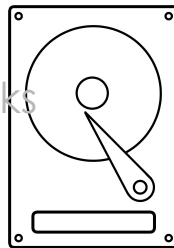
- Implements memtables and SSTables
- Identifies SSTables that should be compacted
- Performs compaction or off-loads it to the StoC



2. Logging Component, LogC, implements logging

3. Storage Component, StoC:

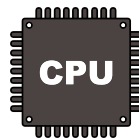
- Stores, retrieves, and manages variable sized blocks
- Manages DRAM, NVMe, Disk, or a hierarchy of these storage devices



Nova-LSM: Components

1. LSM-tree Component, LTC:

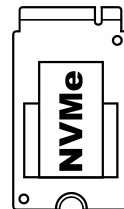
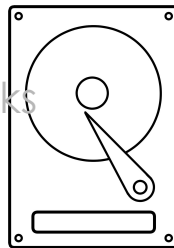
- Implements memtables and SSTables
- Identifies SSTables that should be compacted
- Performs compaction or off-loads it to the StoC



2. Logging Component, LogC, implements logging

3. Storage Component, StoC:

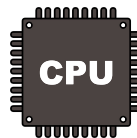
- Stores, retrieves, and manages variable sized blocks
- Manages DRAM, NVMe, Disk, or a hierarchy of these storage devices



Nova-LSM: Components

1. LSM-tree Component, LTC:

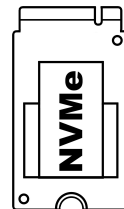
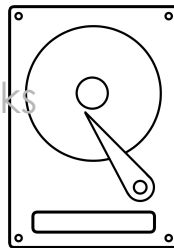
- Implements memtables and SSTables
- Identifies SSTables that should be compacted
- Performs compaction or off-loads it to the StoC



2. Logging Component, LogC, implements logging

3. Storage Component, StoC:

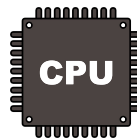
- Stores, retrieves, and manages variable sized blocks
- Manages DRAM, NVMe, Disk, or a hierarchy of these storage devices



Nova-LSM: Components

1. LSM-tree Component, LTC:

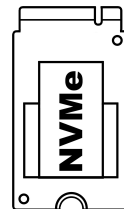
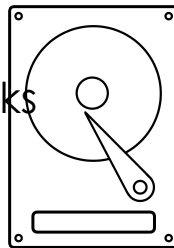
- Implements memtables and SSTables
- Identifies SSTables that should be compacted
- Performs compaction or off-loads it to the StoC



2. Logging Component, LogC, implements logging

3. Storage Component, StoC:

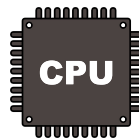
- Stores, retrieves, and manages variable sized blocks
- Manages DRAM, NVMe, Disk, or a hierarchy of these storage devices



Nova-LSM: Components

1. LSM-tree Component, LTC:

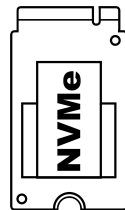
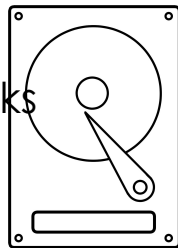
- Implements memtables and SSTables
- Identifies SSTables that should be compacted
- Performs compaction or off-loads it to the StoC



2. Logging Component, LogC, implements logging

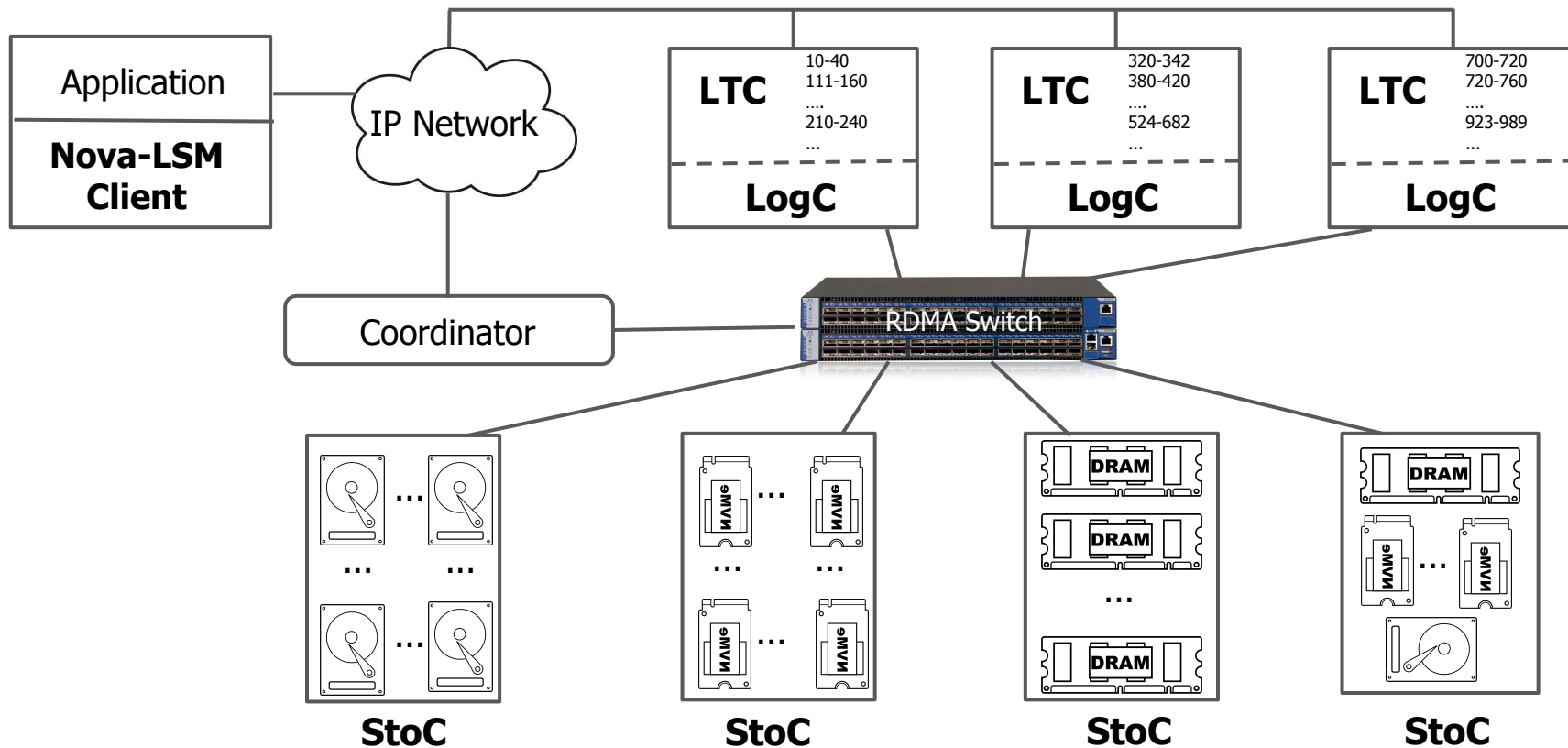
3. Storage Component, StoC:

- Stores, retrieves, and manages variable sized blocks
- Manages DRAM, NVMe, Disk, or a hierarchy of these storage devices



Nova-LSM may be extended with additional components, e.g., a cache manager.

Nova-LSM: Architecture

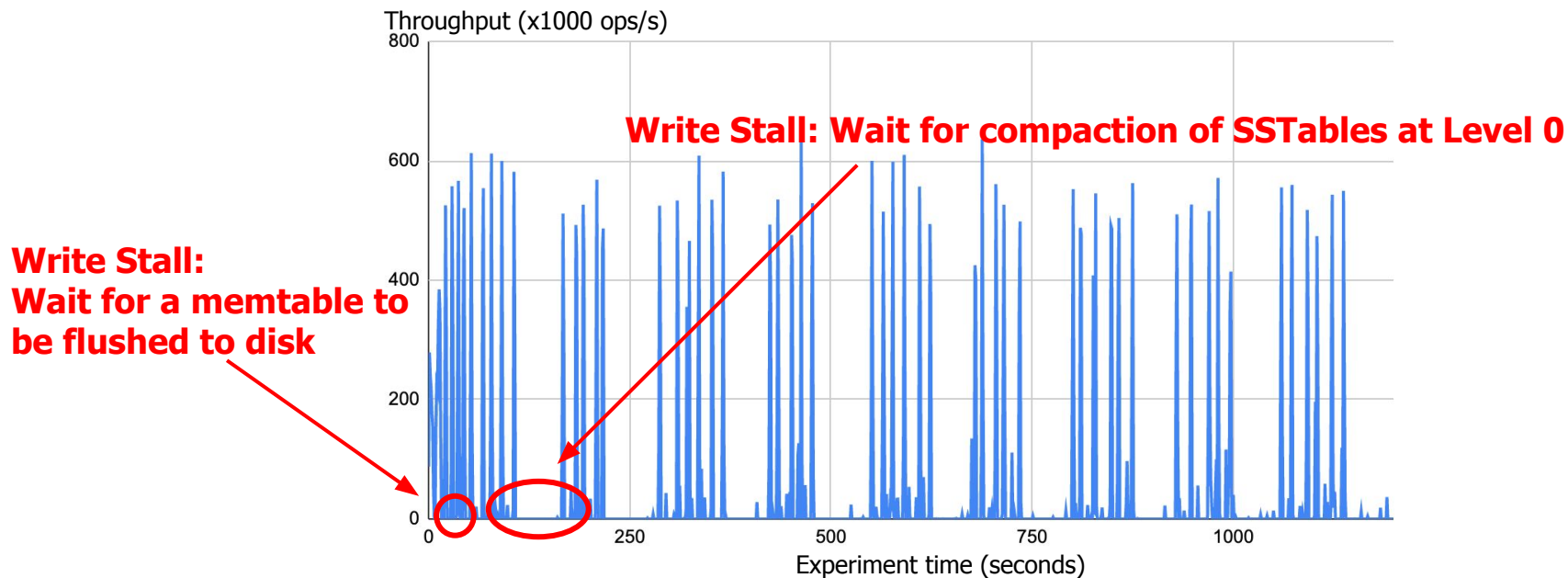


Challenges

1. Write stalls.
2. Large memory slows down gets.
3. Large memory slows down scans.
4. Durable logging is insufficient to achieve high availability.

Challenge: Write Stalls

- Writes are stalled [1,2,3] when
 - all memtables are full,
 - or the number of SSTables at Level 0 exceeds a threshold, e.g., 10.



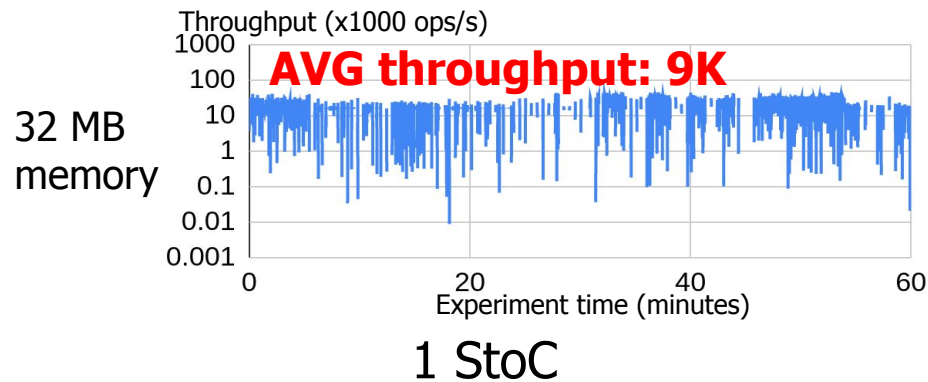
[1]. Chen Luo and Michael J. Carey. 2019. On Performance Stability in LSM-Based Storage Systems. Proc. VLDB Endow. 13, 4 (Dec. 2019).

[2]. Oana Balmau, Florin Dinu, Willy Zwaenepoel, Karan Gupta, Ravishankar Chandhiramoorthi, and Diego Didona. 2019. SILK: Preventing Latency Spikes in Log-Structured Merge Key-Value Stores. In 2019 USENIX Annual Technical Conference (USENIX ATC 19).

[3]. Russell Sears and Raghu Ramakrishnan. 2012. BLSM: A General Purpose Log Structured Merge Tree. In Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data (Scottsdale, Arizona, USA) (SIGMOD '12).

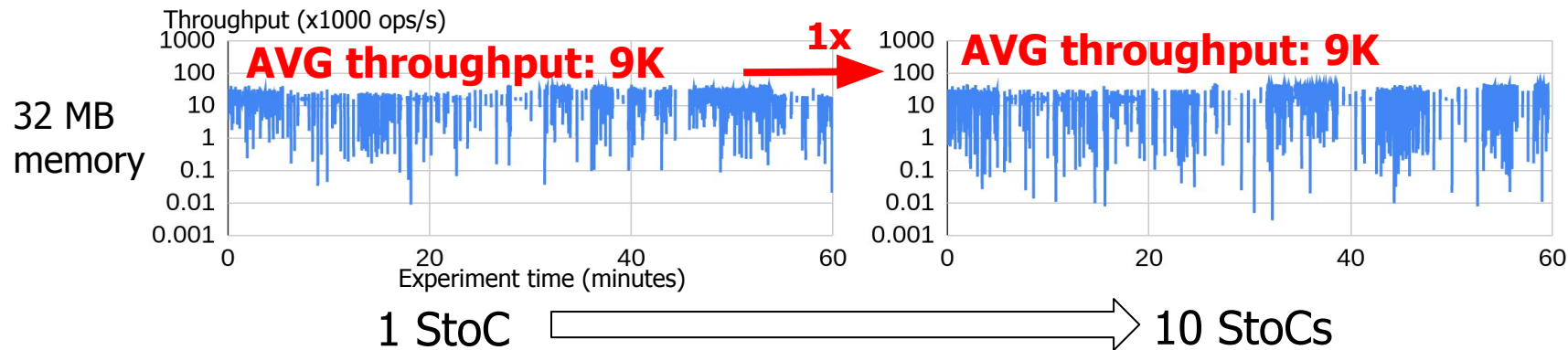
Solution: Write Stalls

- Vertical scaling: Use large memory.
- Horizontal scaling: Use the bandwidth of many StoCs.



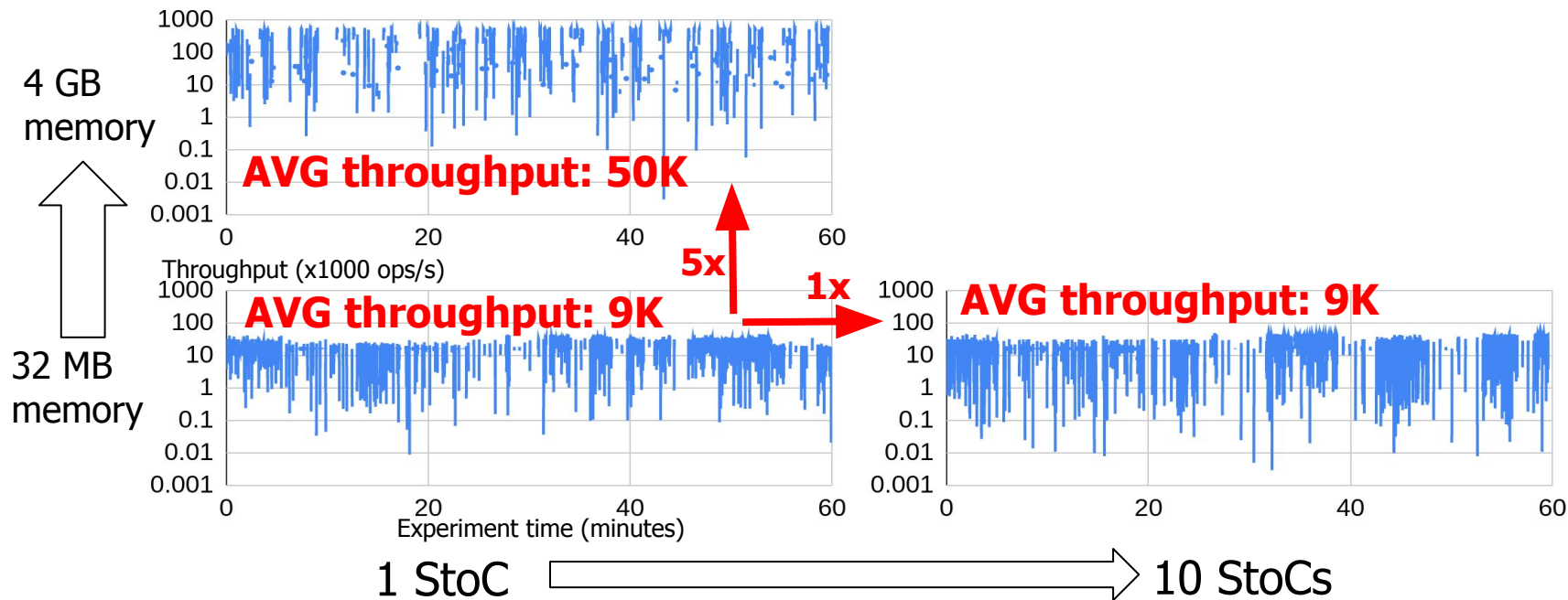
Solution: Write Stalls

- Vertical scaling: Use large memory.
- Horizontal scaling: Use the bandwidth of many StoCs.



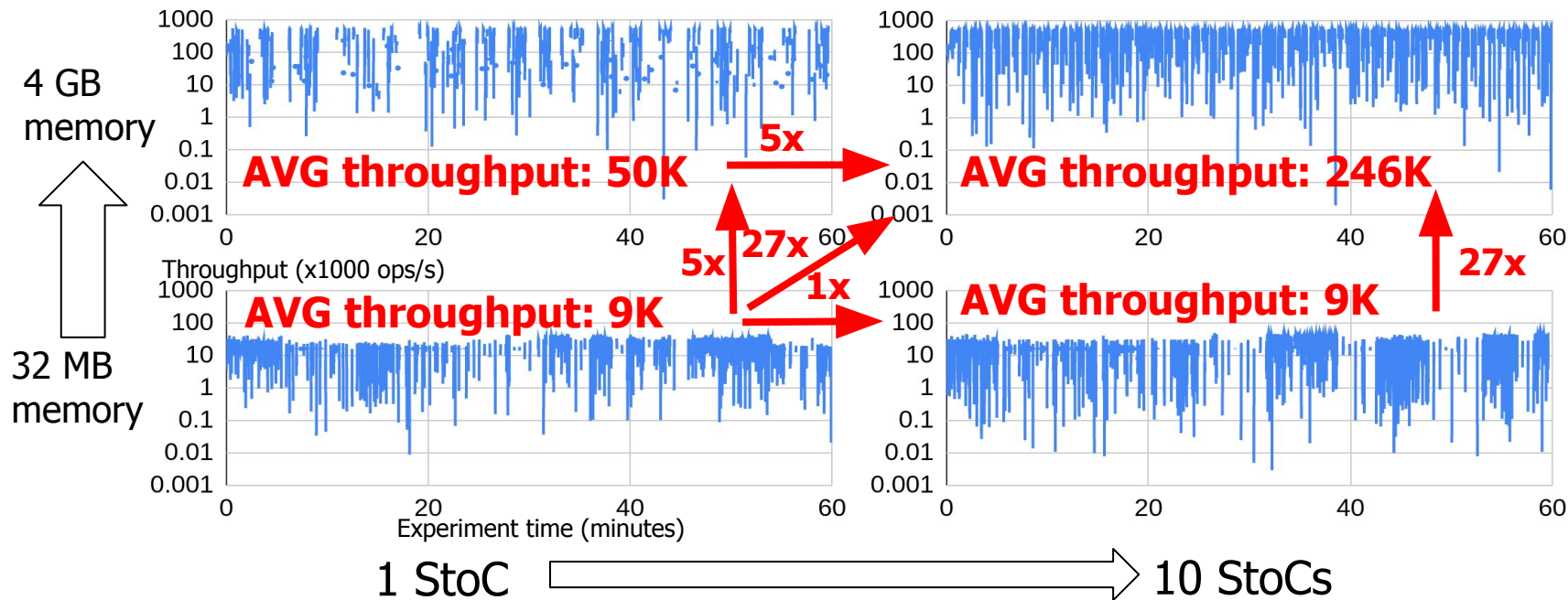
Solution: Write Stalls

- Vertical scaling: Use large memory.
- Horizontal scaling: Use the bandwidth of many StoCs.



Solution: Write Stalls

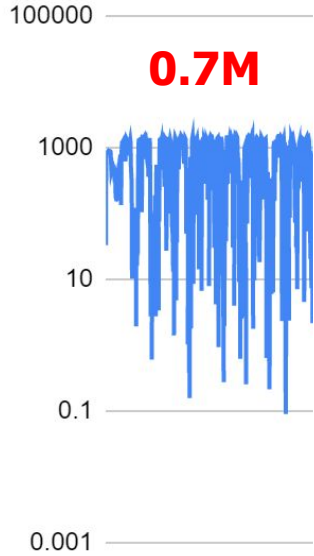
- Vertical scaling: Use large memory.
- Horizontal scaling: Use the bandwidth of many StoCs.



Scale Resources Independently

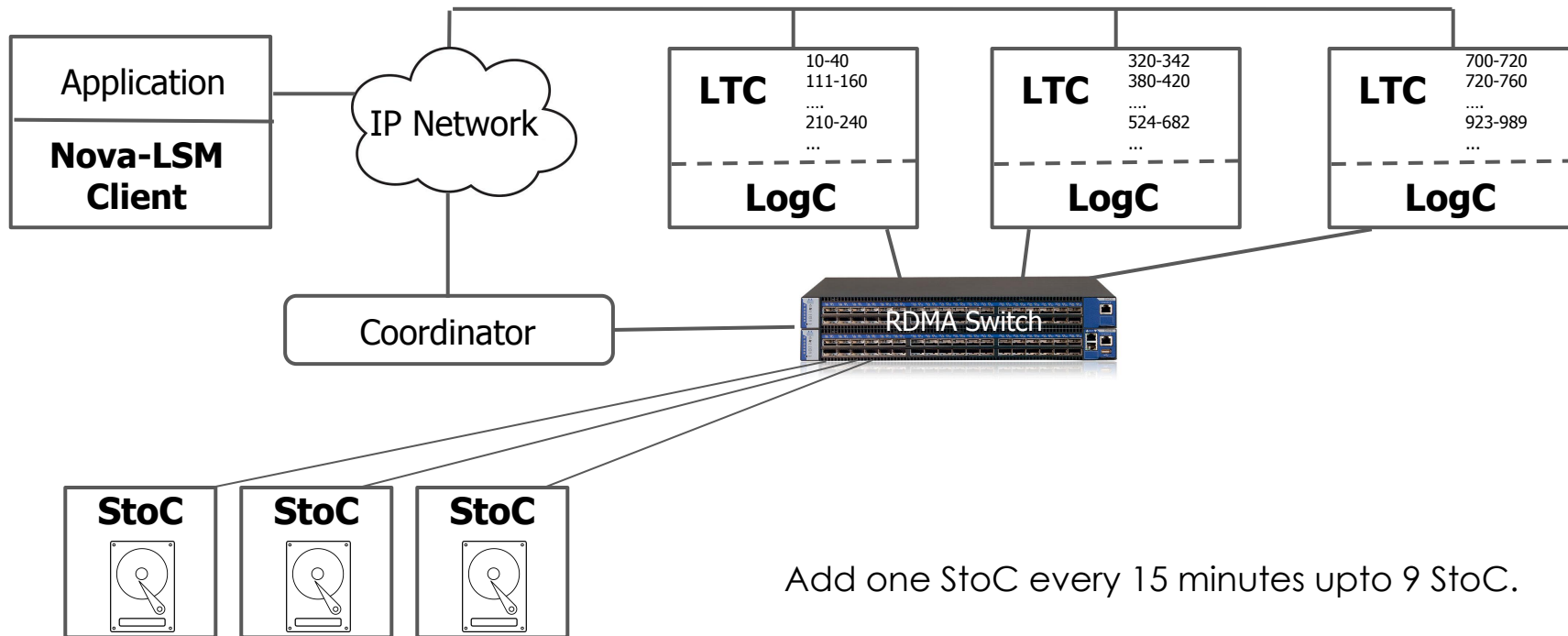
Throughput (x1000 ops/s)

AVG throughput

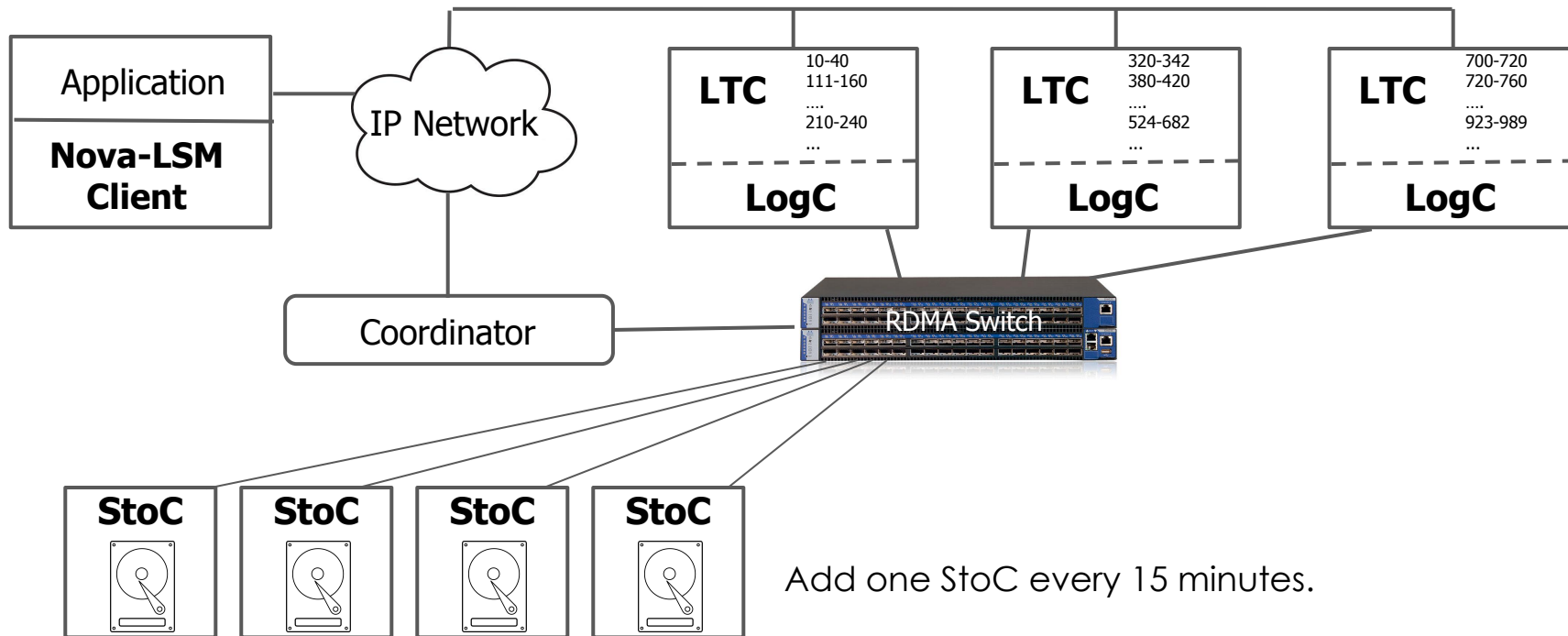


Experiment time (minutes)

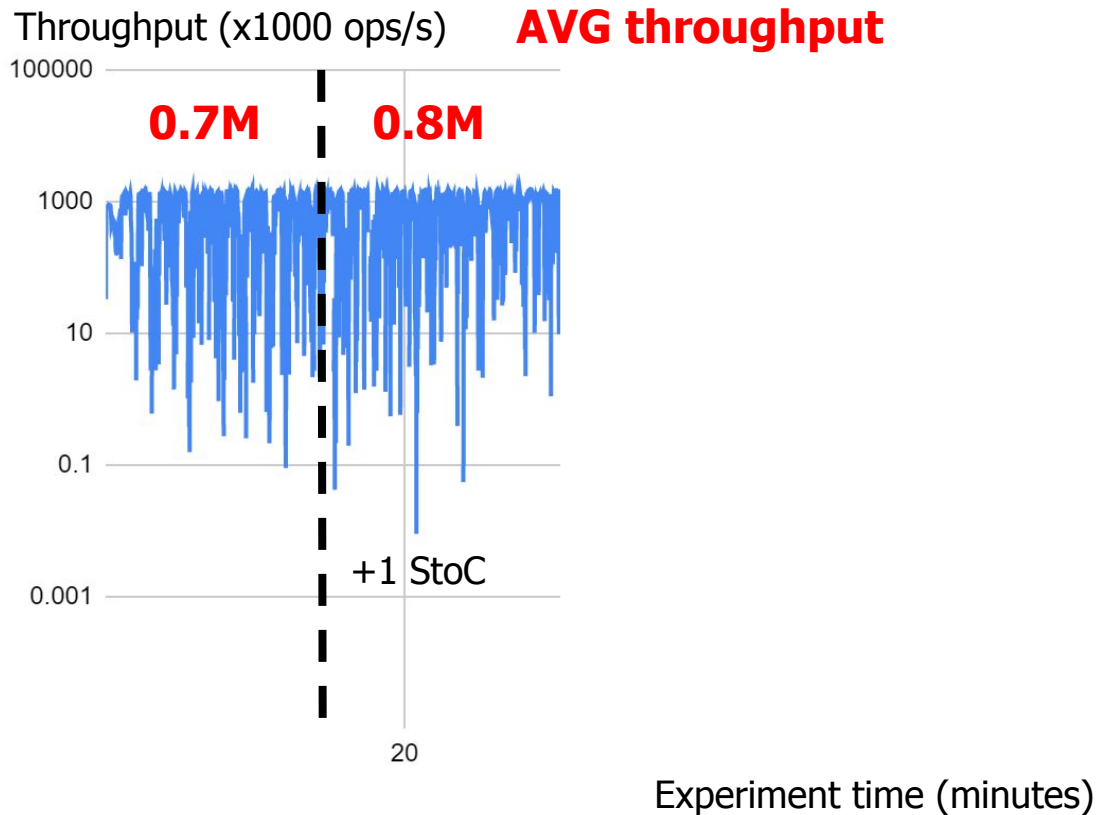
Nova-LSM: Elastic Storage Layer



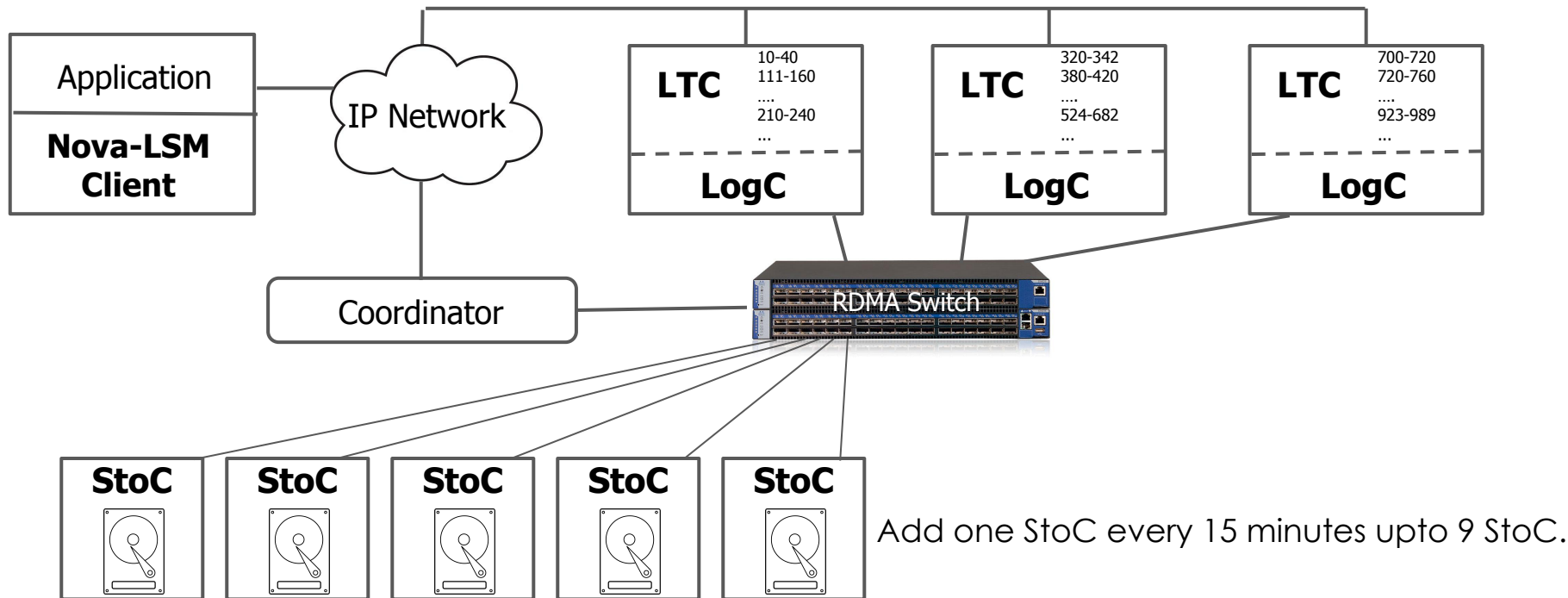
Nova-LSM: Elastic Storage Layer



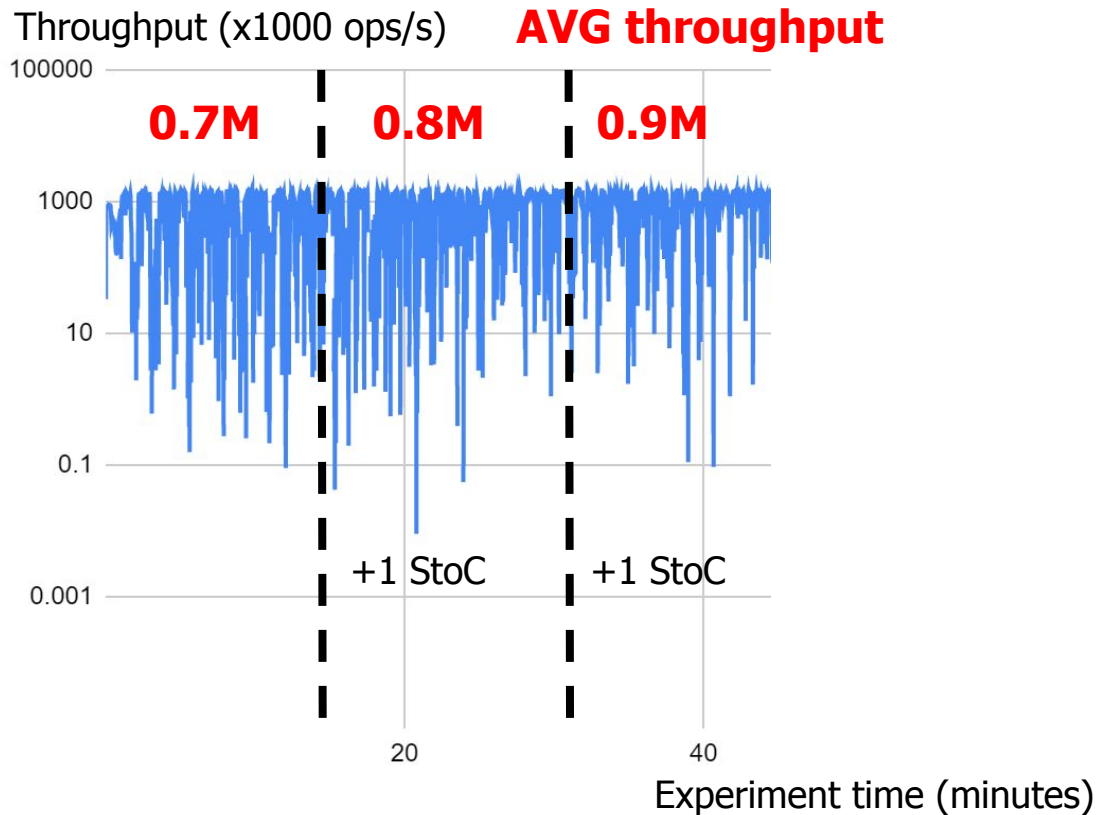
Scale Resources Independently



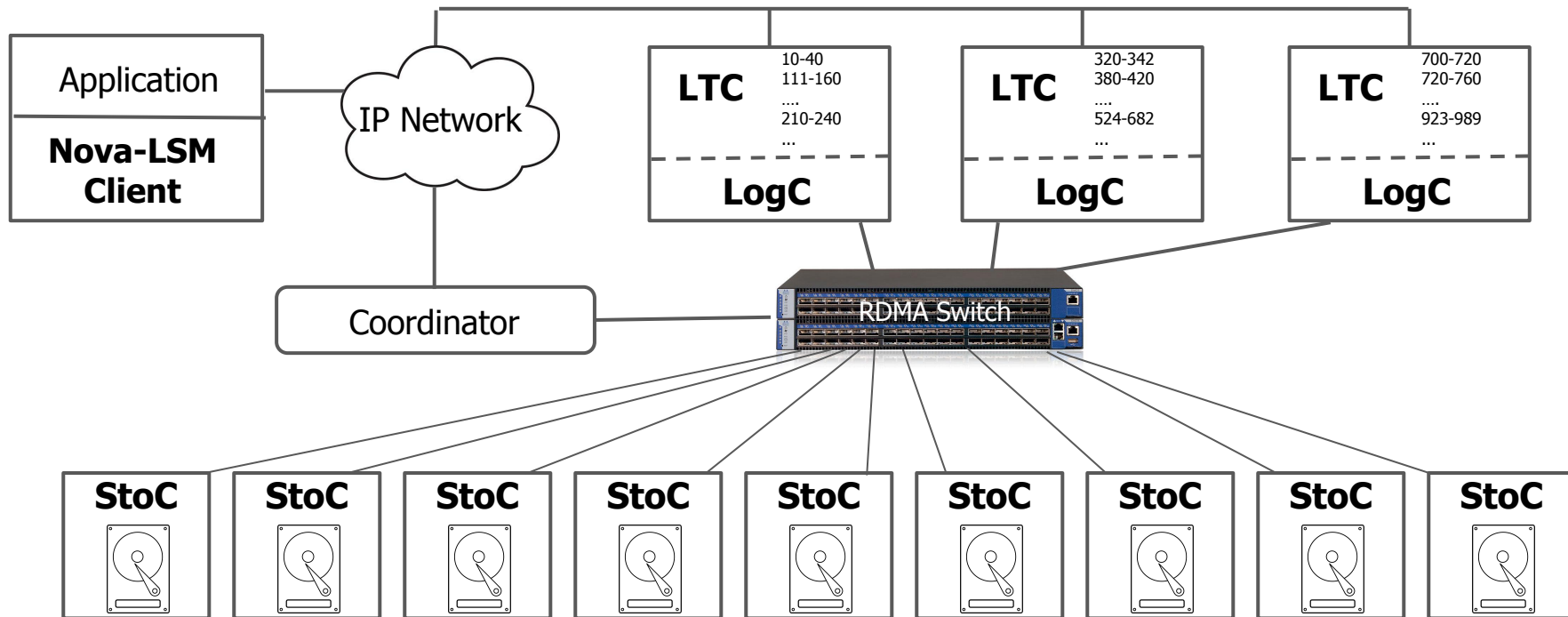
Nova-LSM: Elastic Storage Layer



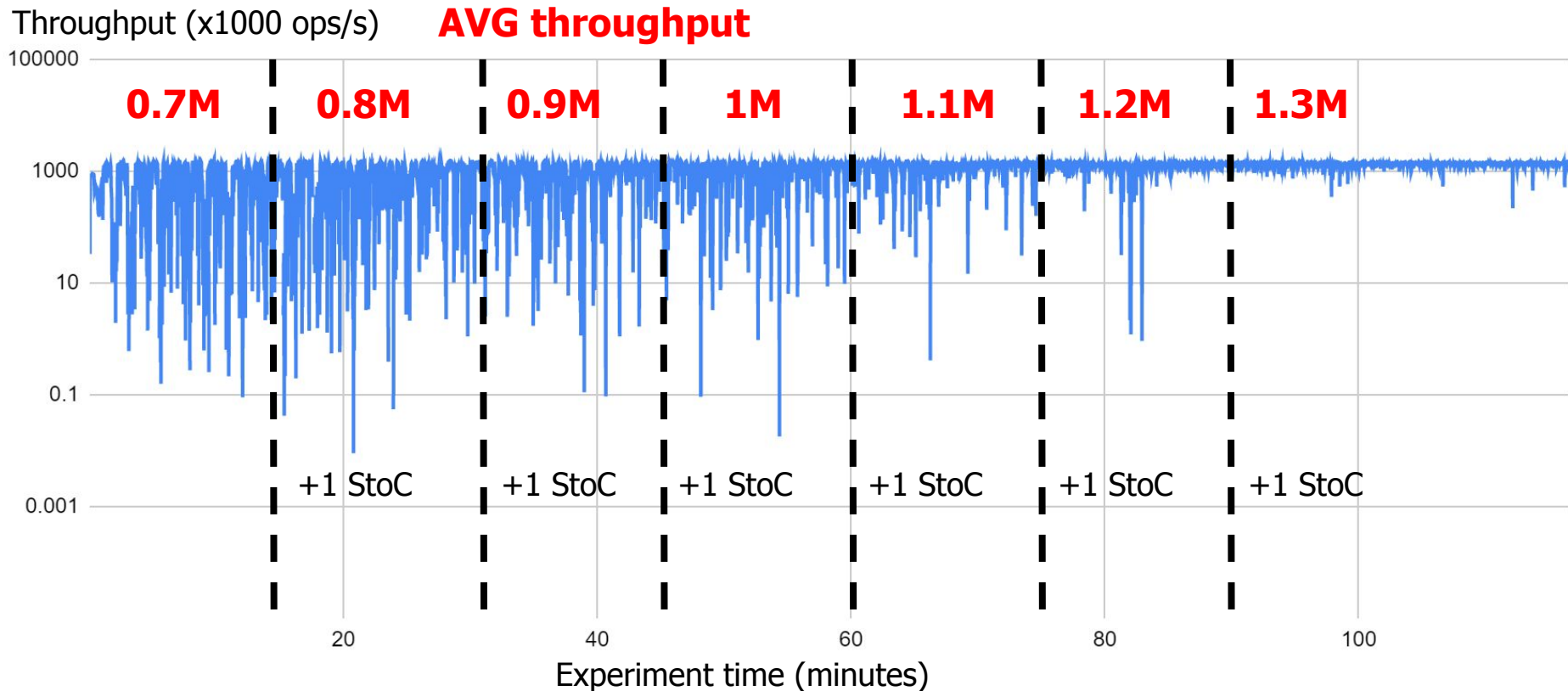
Scale Resources Independently



Nova-LSM: Elastic Storage Layer

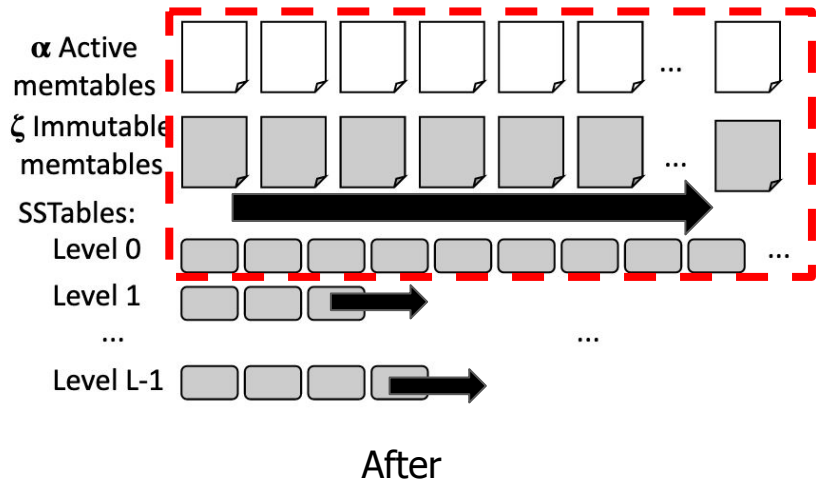
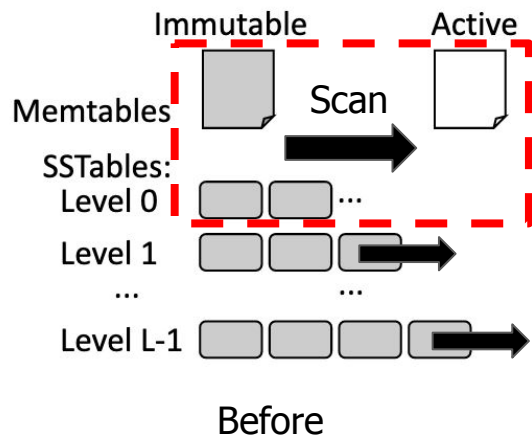


Scale Resources Independently



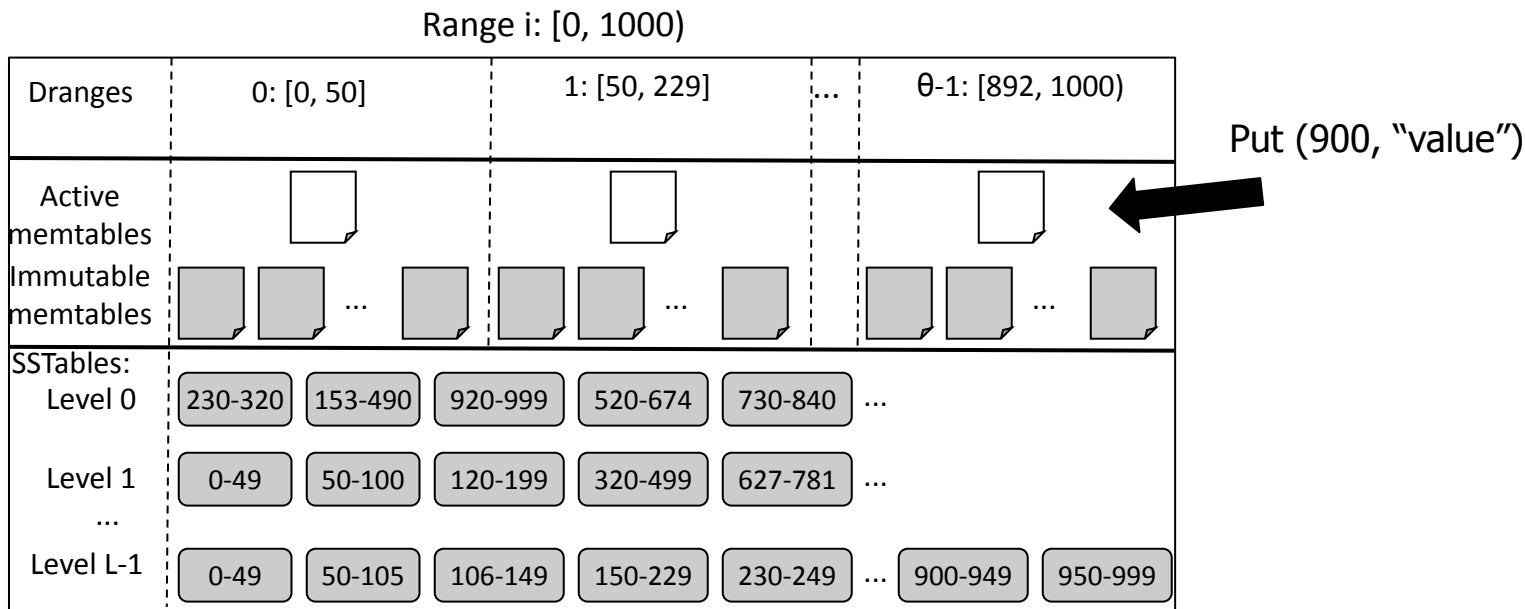
Challenge II: Scans are slowed down

- Large memory slows down scans.

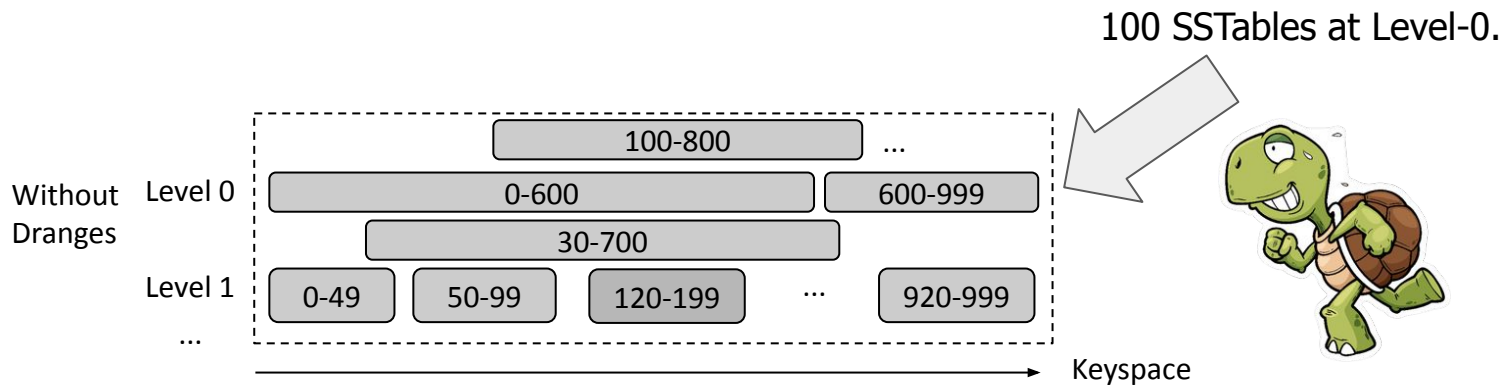


LTC: Dynamic Ranges (Drange)

- Construct dynamic ranges at runtime based on workload.
 - Compact SStables in parallel.
 - Gets/Scans search memtables of the corresponding Dranges.
 - Multiple active memtables process concurrent writes using multiple cores.



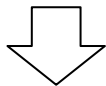
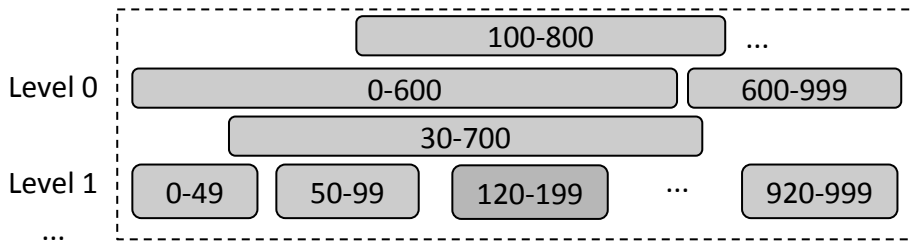
Nova-LSM: Dranges



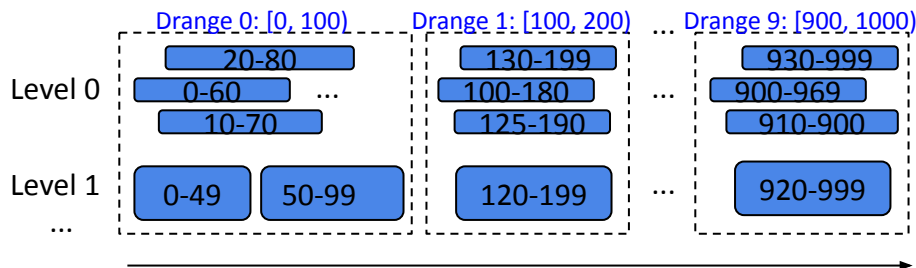
Nova-LSM: Dranges

- Drange facilitates parallel compaction.
 - A factor of 2.3x to 26x throughput improvement depending on workload.

Without
Dranges



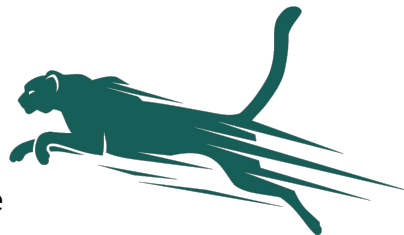
With
Dranges



Keyspace

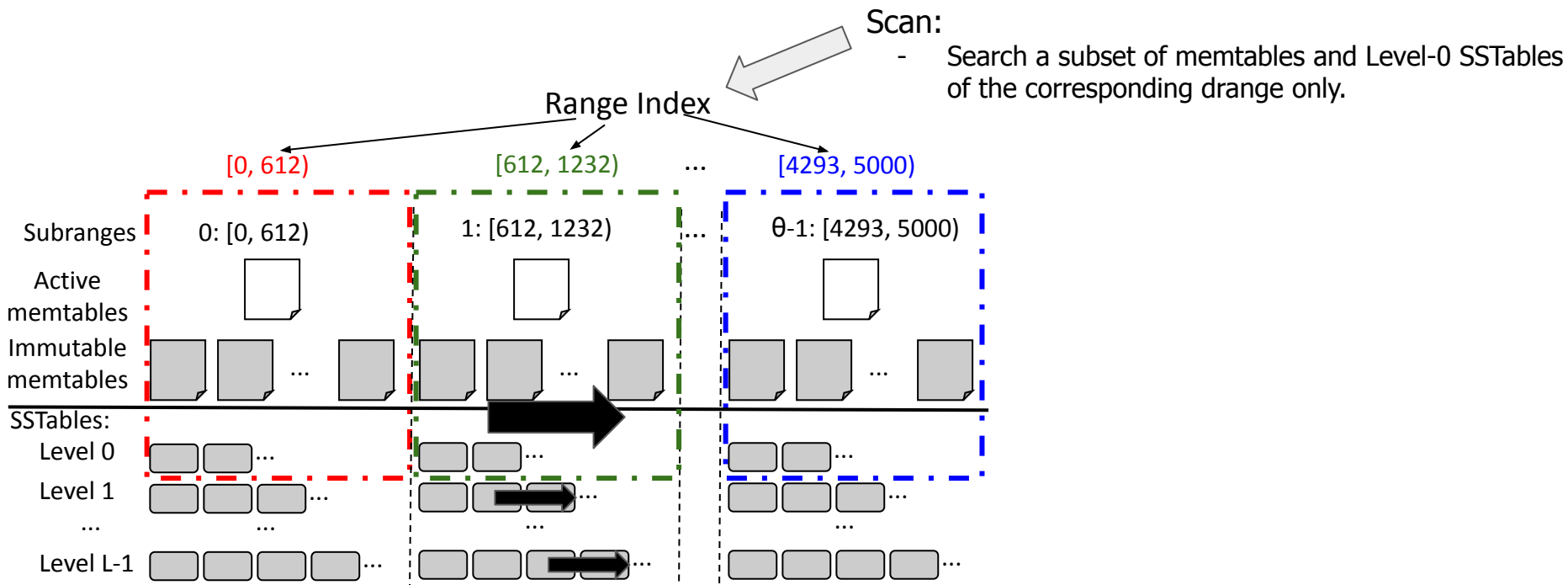


100 overlapping
SSTables at Level-0
reduces to 10.

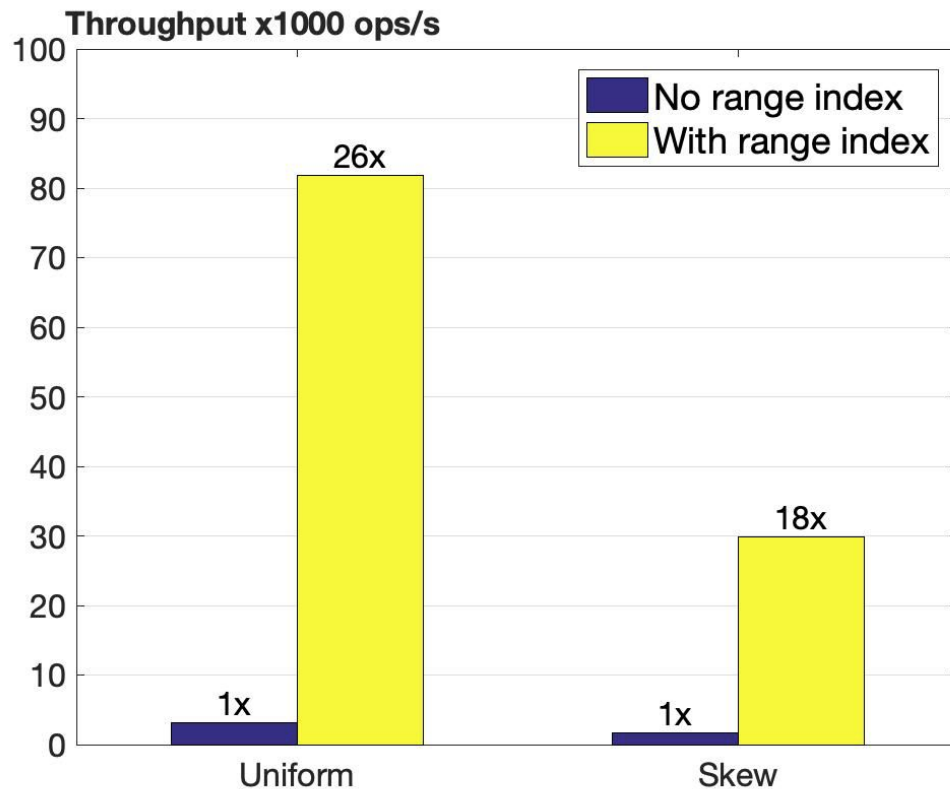


Solution to Challenge II: Divide and Conquer

- Construct range index dynamically.

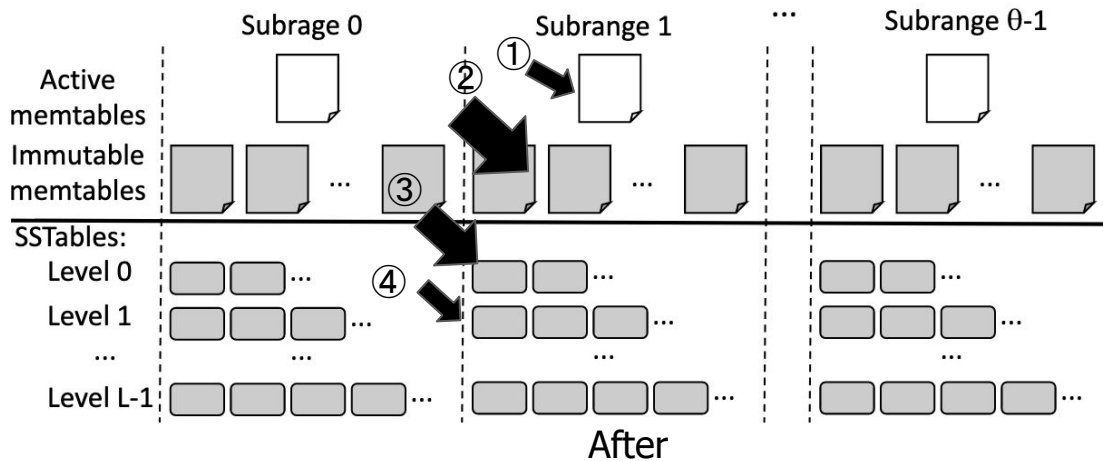
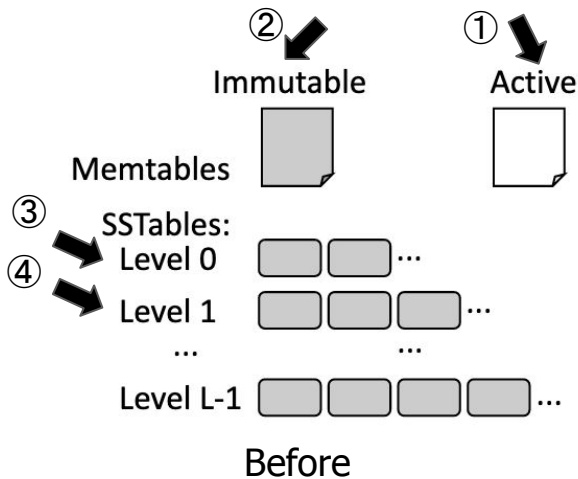


Evaluation: Range Index



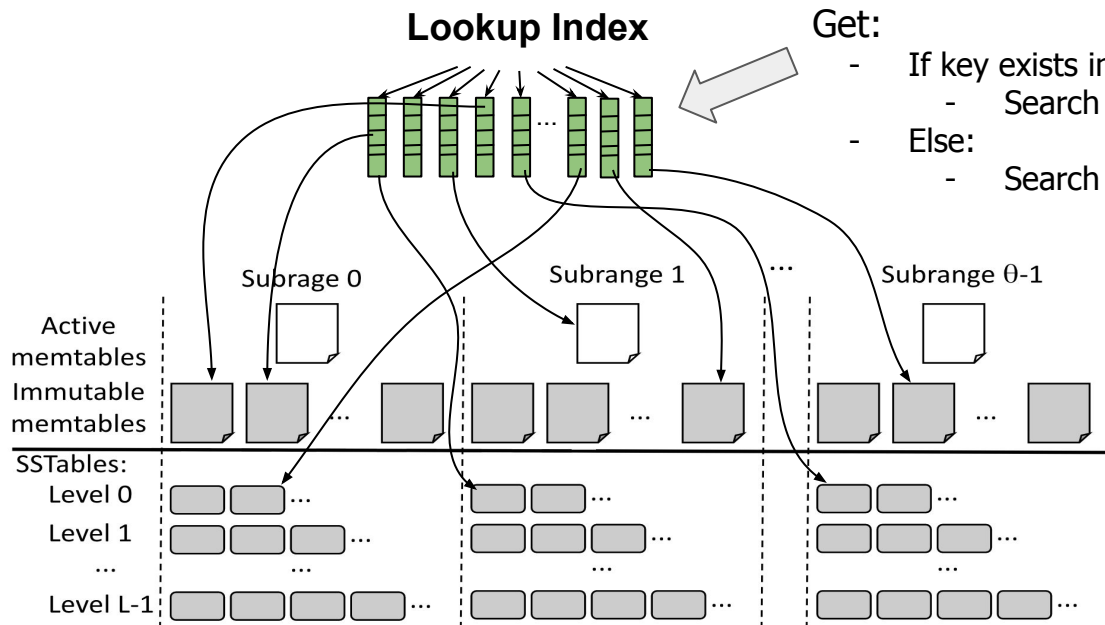
Challenge III: Gets are slowed down

- Large memory slows down gets.

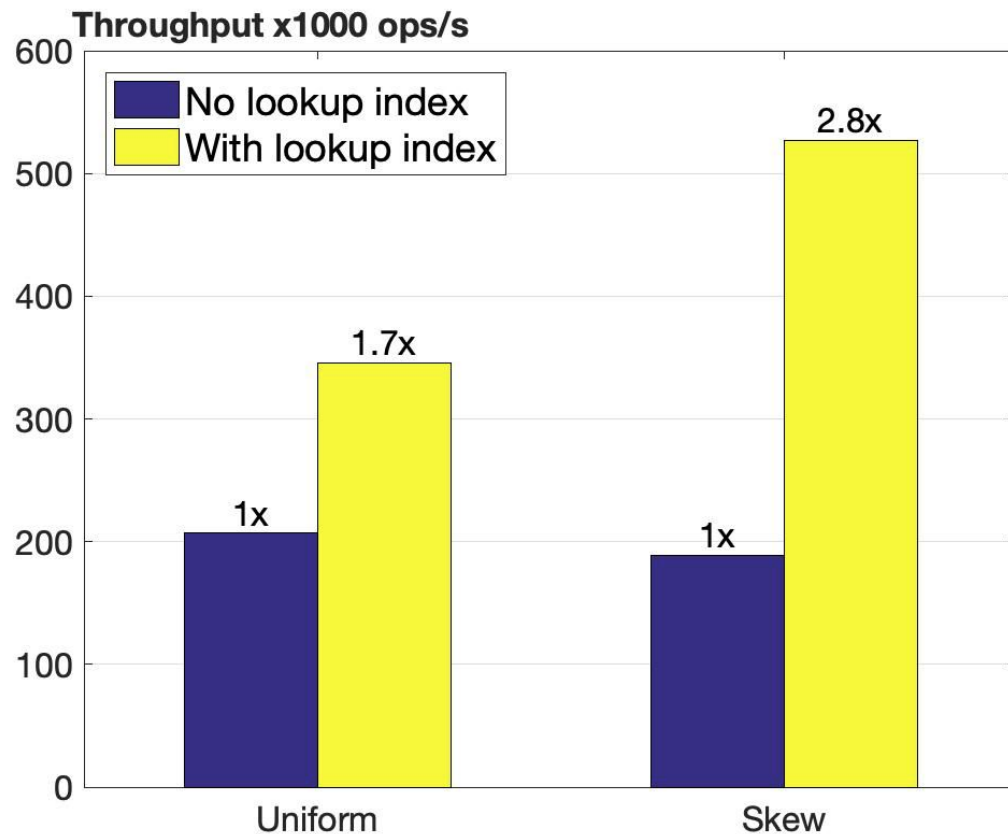


Solution to Challenge III: Lookup Index

- Use lookup index.

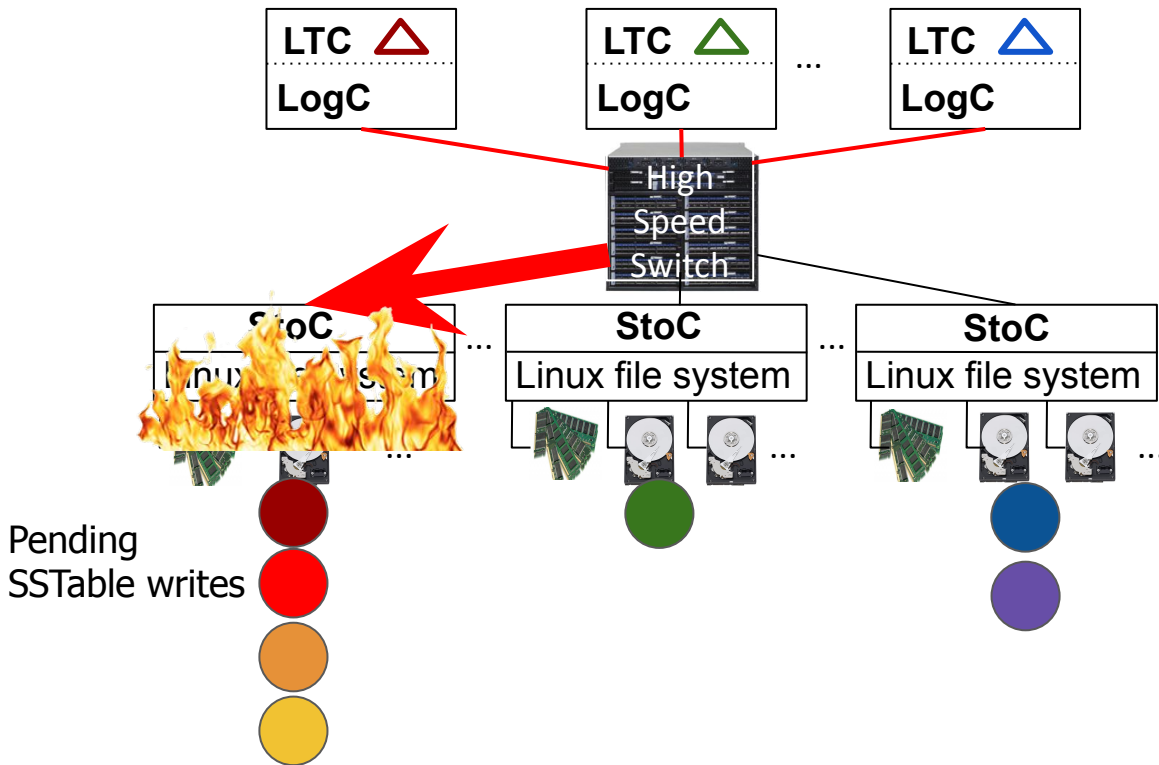


Evaluation: Lookup Index



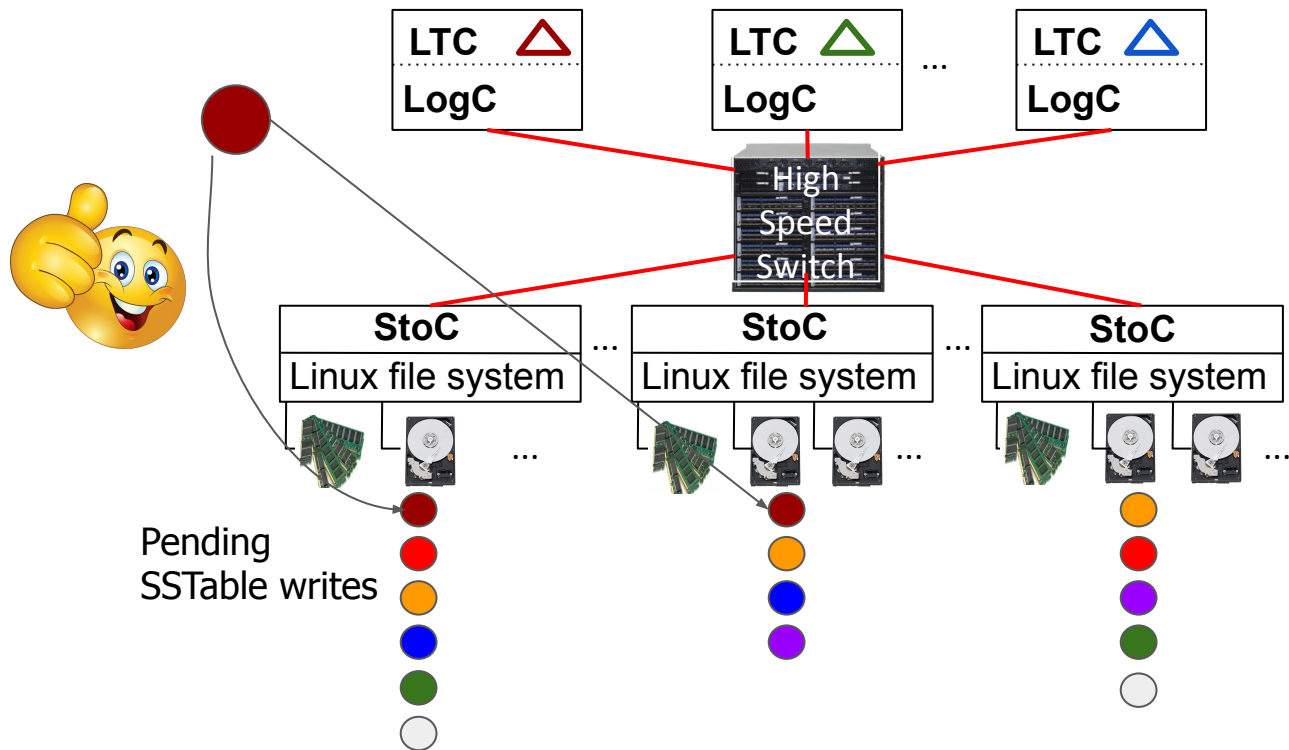
Challenge IV: Temporary Bottlenecks

- Random requests collide and reference the same StoC.

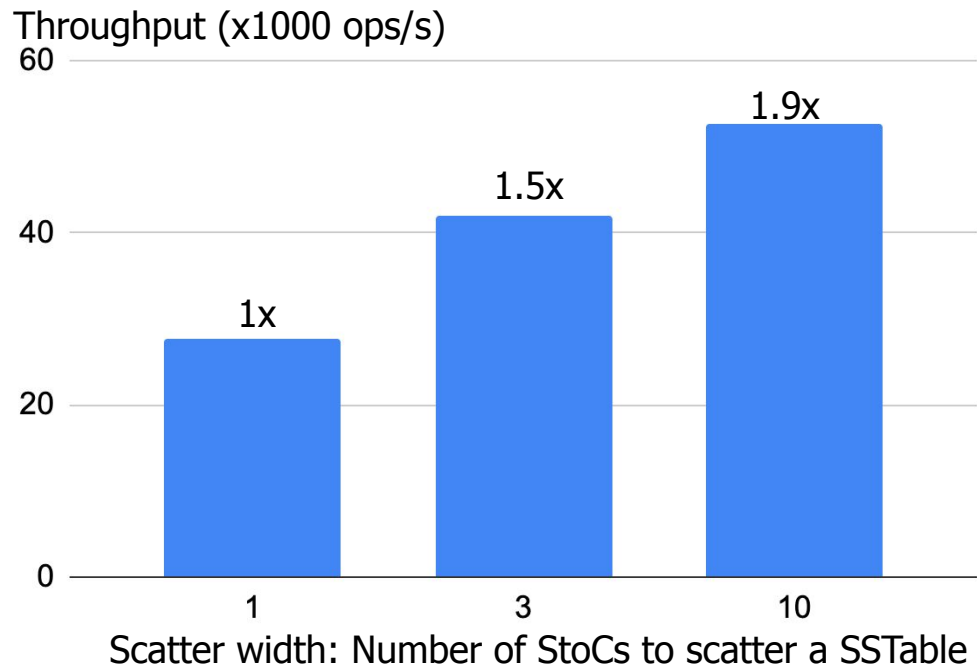


Solution to Challenge IV: Partition SSTables

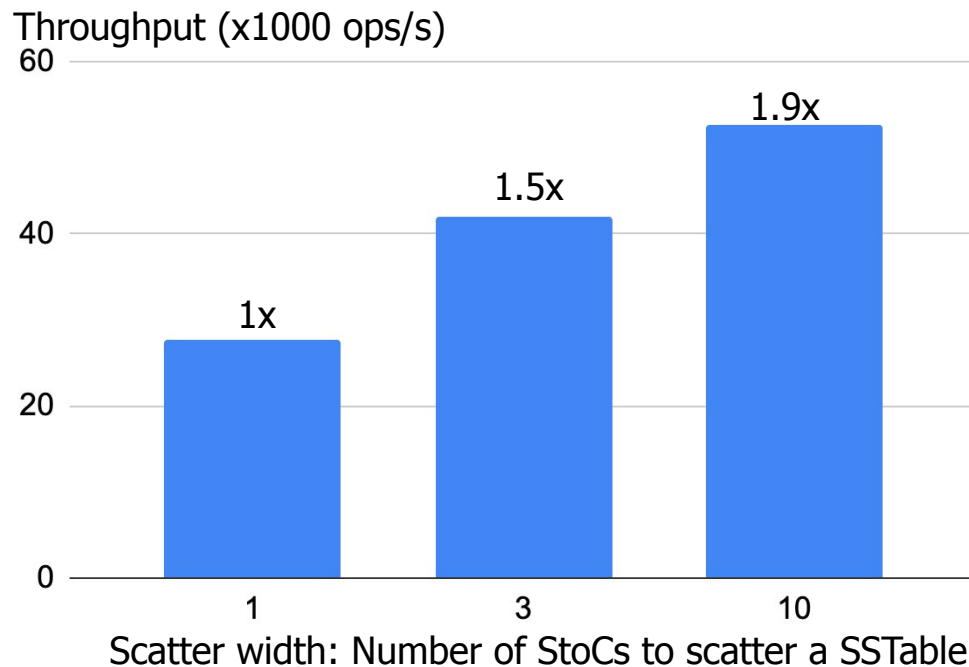
- Scatter blocks of a SSTable across multiple StoCs.



Evaluation: Partition SSTables



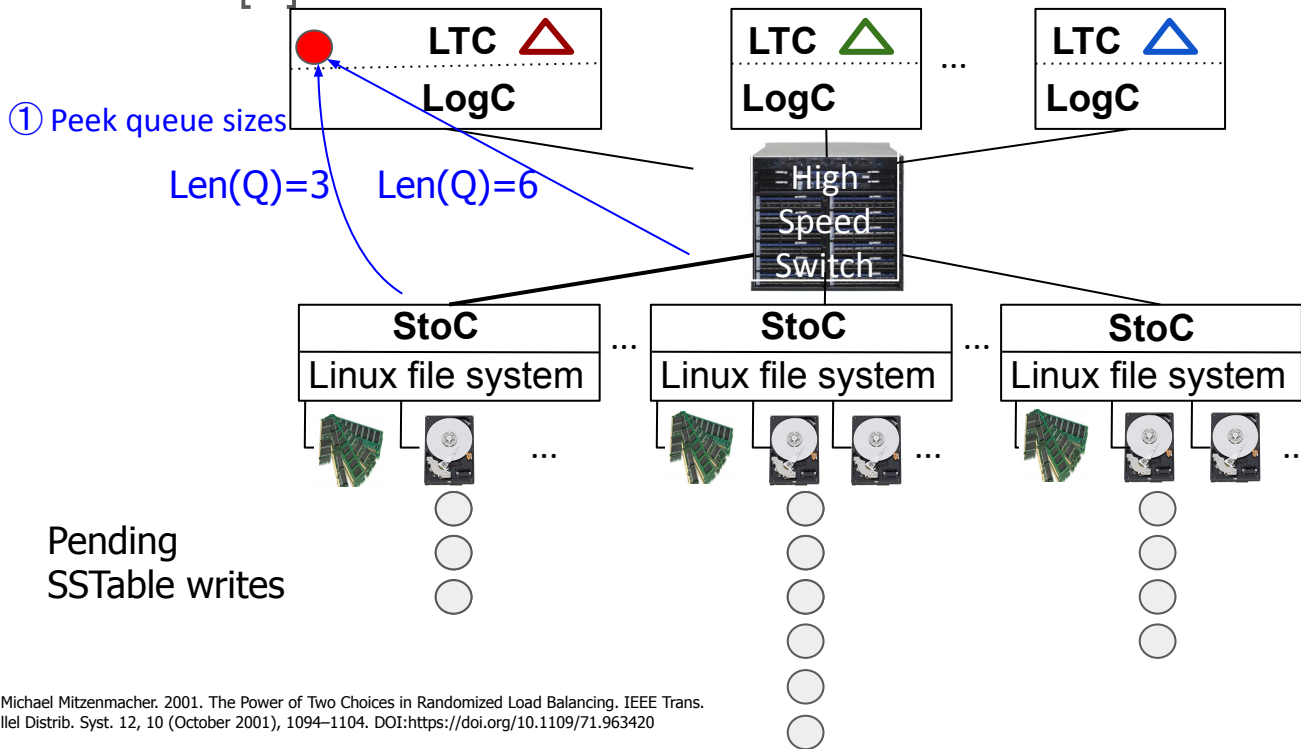
Evaluation: Partition SSTables



Disadvantage: Disk seeks and rotational latencies waste disk bandwidth.

Solution to Challenge IV: Power-of-d

- Observation: Writes may be performed using any StoC.
- Power-of-d [2].



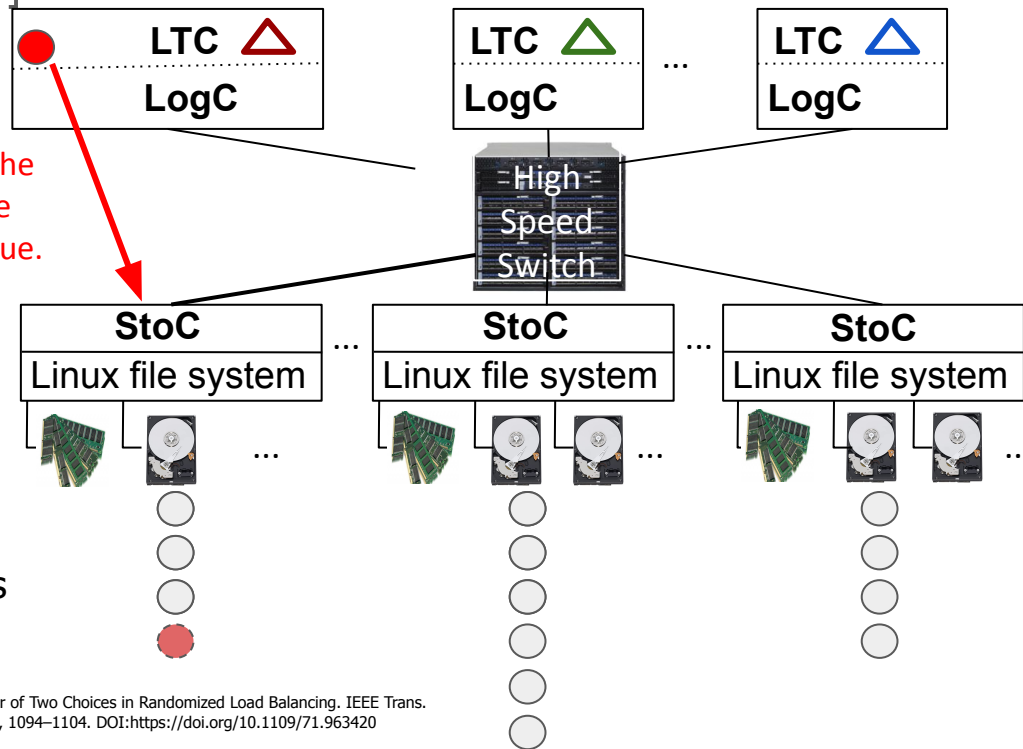
Solution to Challenge IV: Power-of-d

- Observation: Writes may be performed using any StoC.
- Power-of-d [2].

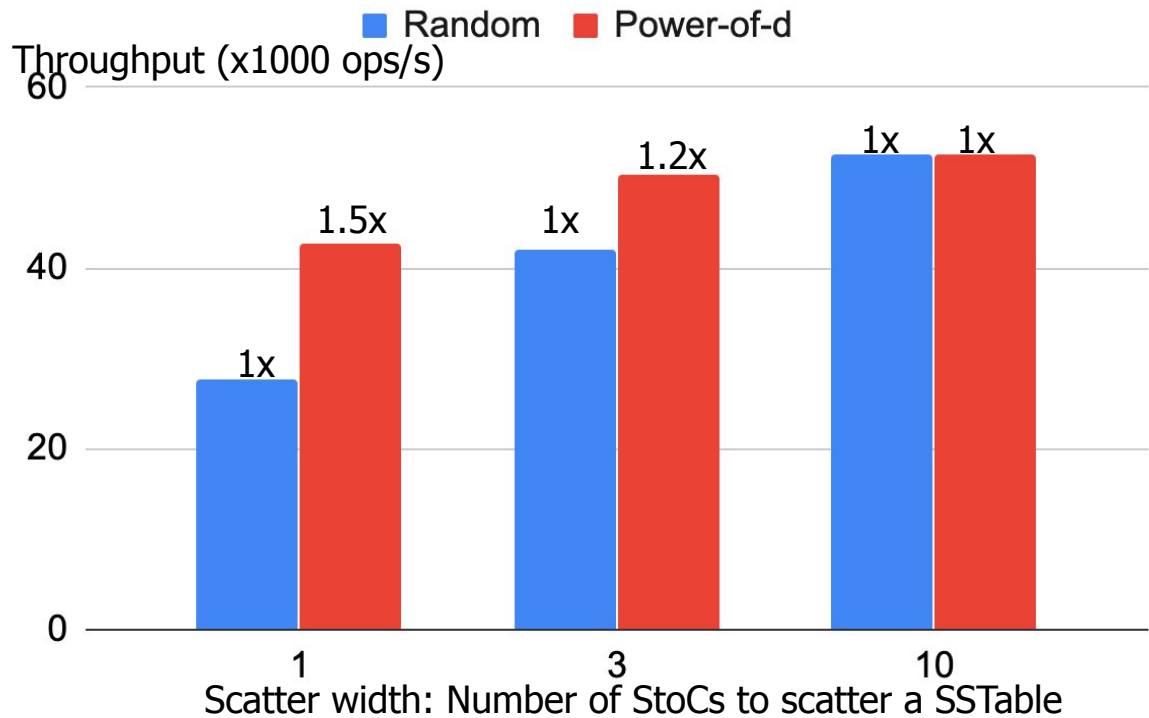
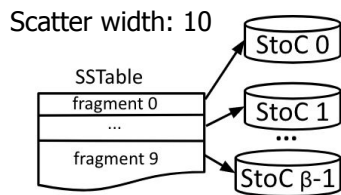
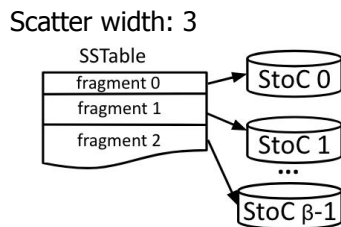
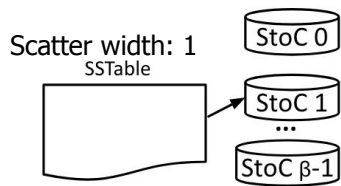


② Write to the
StoC with the
shortest queue.

Pending
SSTable writes

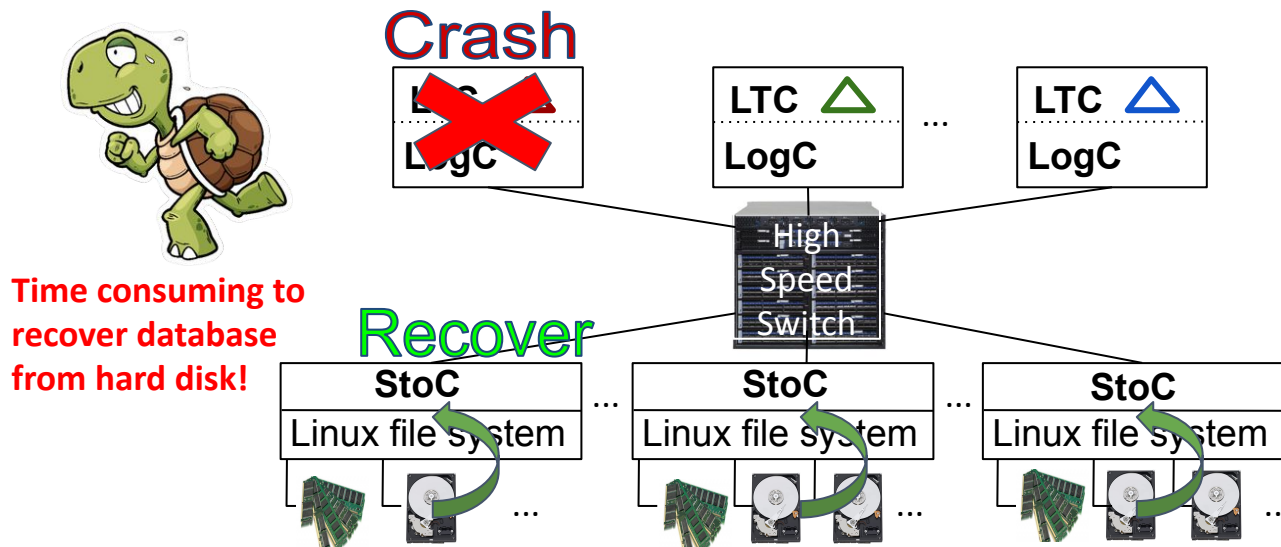


Evaluation: Power-of-d



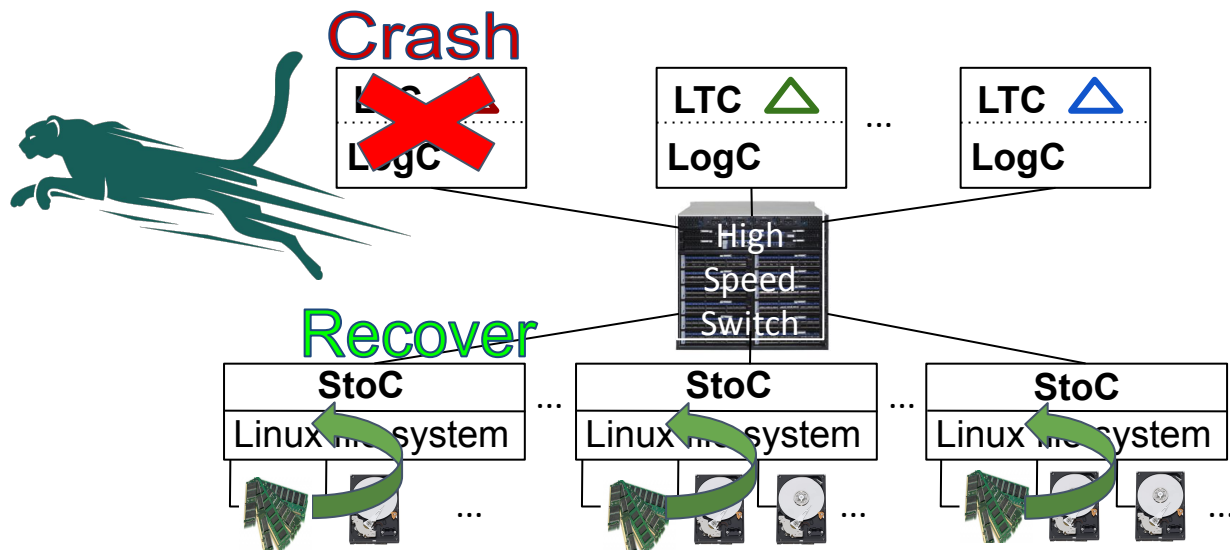
Challenge IV: Logging

- Cloud infrastructure is not the same as on-premise infrastructure. It is more available [5].
- Separate availability from durability.



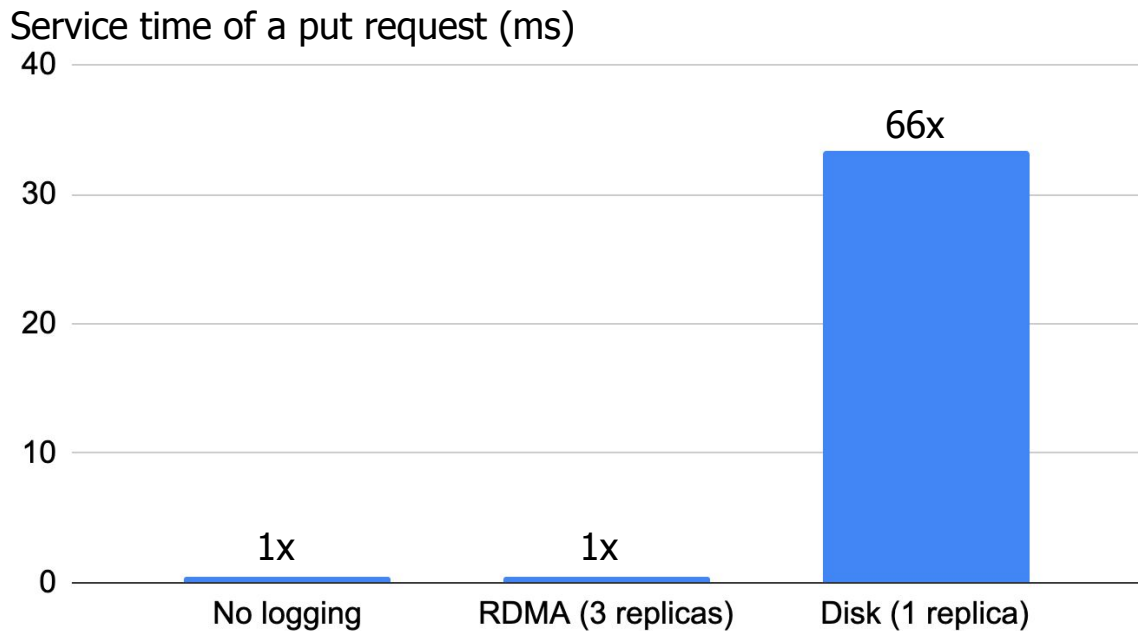
Solution to Challenge IV: Replicating Log Records

- Replicate log records in the memory of StoCs to provide high availability.
 - Disadvantage: Consume more memory and may lose data if all replicas fail.



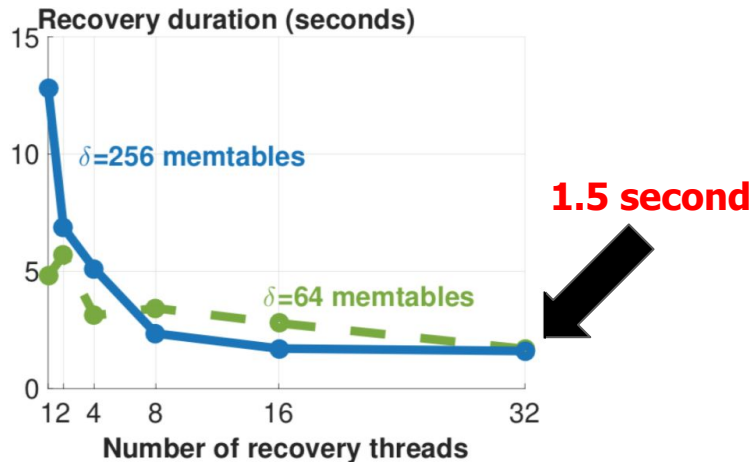
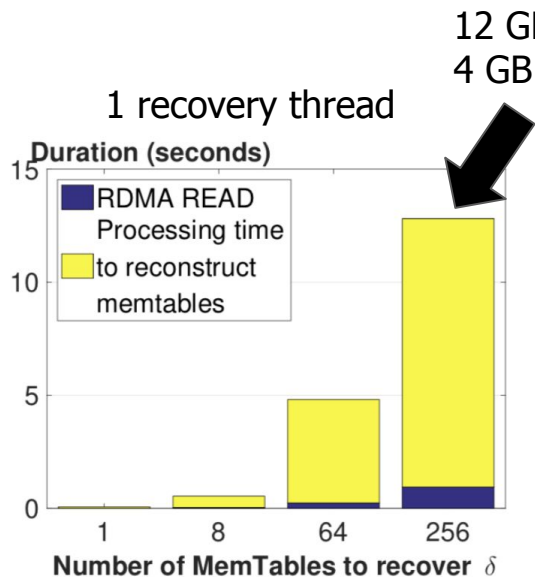
Evaluation: Logging

- LogC uses one RDMA WRITE to append a log record.



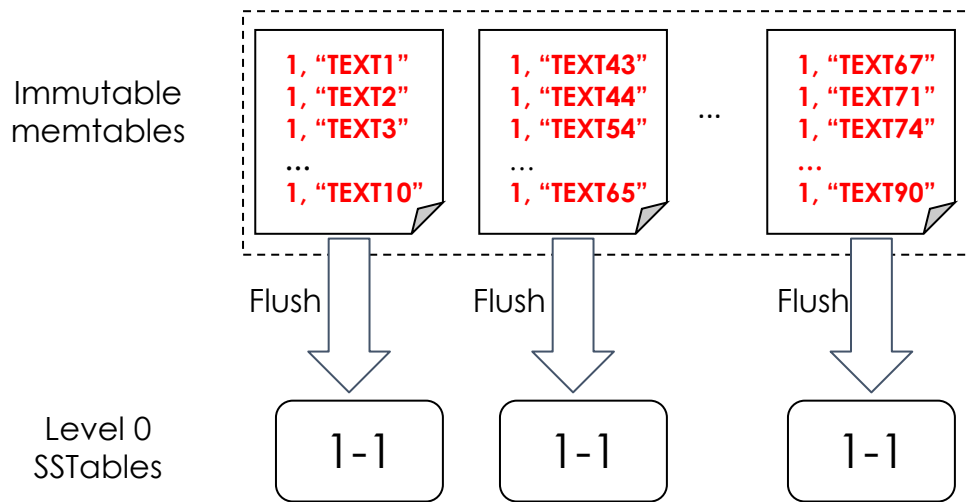
Evaluation: Recovery

- LogC uses one RDMA READ to fetch all log records of a log file.



Challenge-VI: Skewed Access Pattern

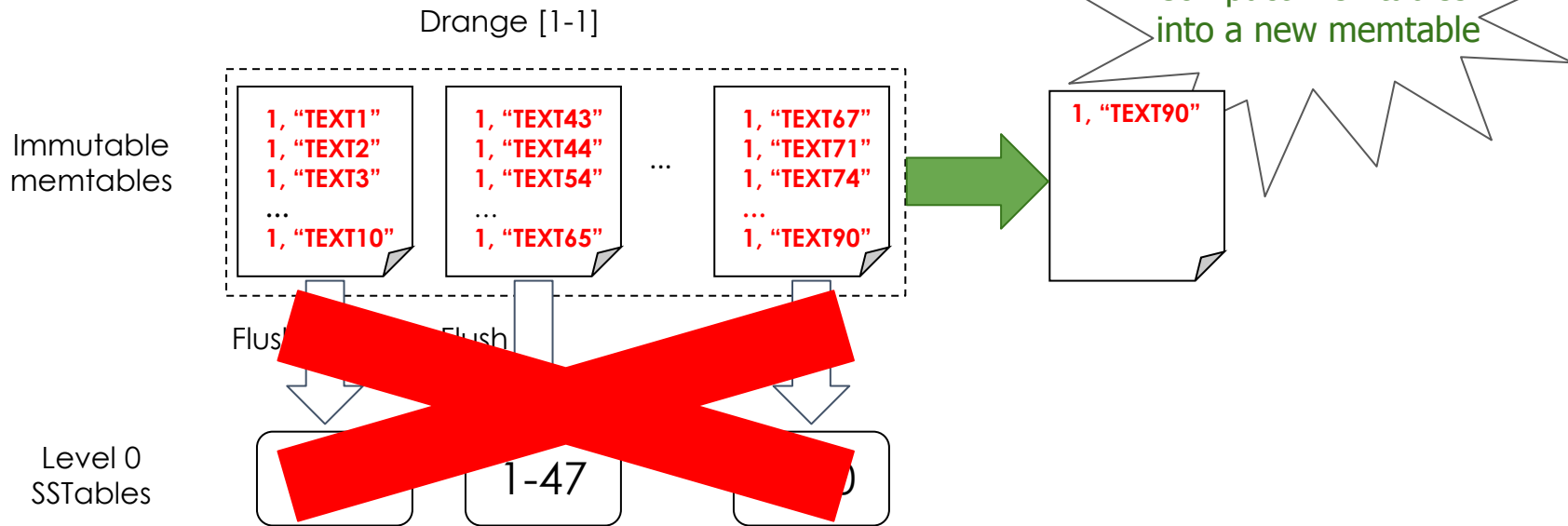
Without Dranges



**Write all versions
of key 1 to disk**

Solution

- Dranges enable LTC to write 65% less data to StoCs with skewed data access.
 - An additional 1.5x to 3x throughput improvement.



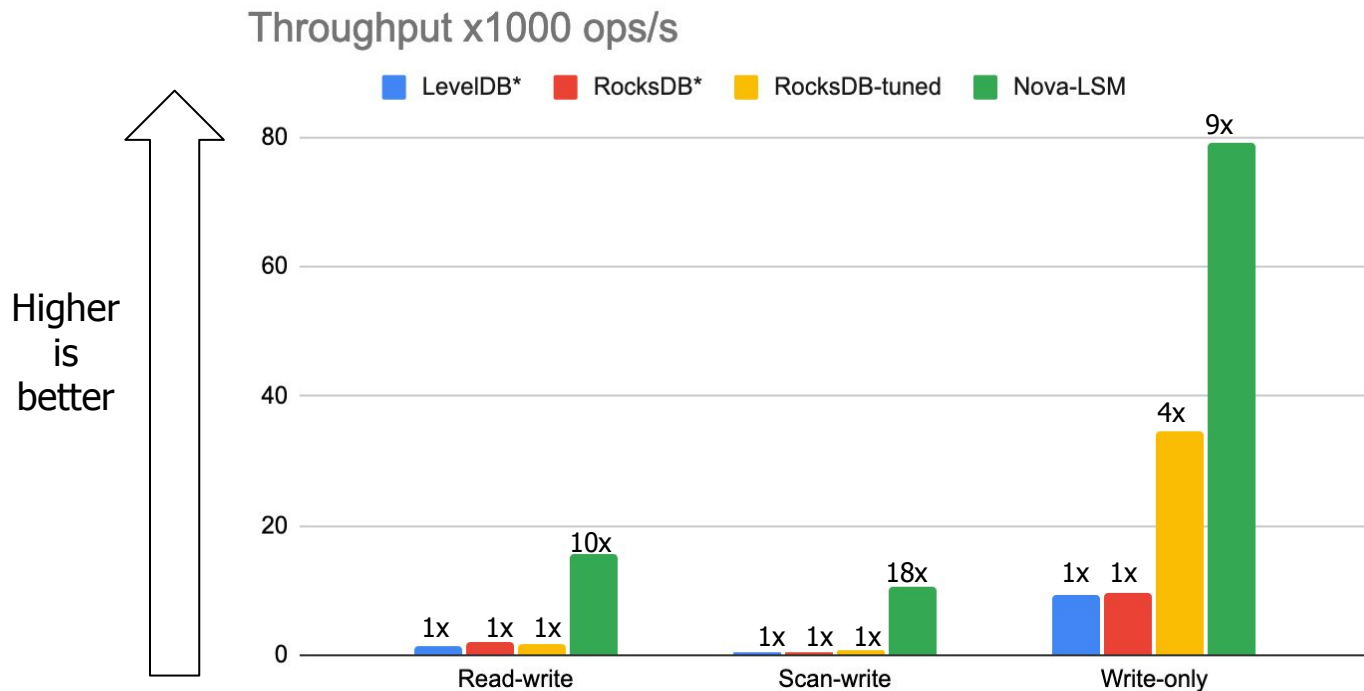
Evaluation

- A 2 TB database (2 billion records) partitioned across 10 nodes.
- Access distribution: Skewed.
- YCSB Workloads:

Workload	Actions
Read-write	50% Read, 50% Write
Scan-write	50% Scan, 50% Write
Write-only	100% Write

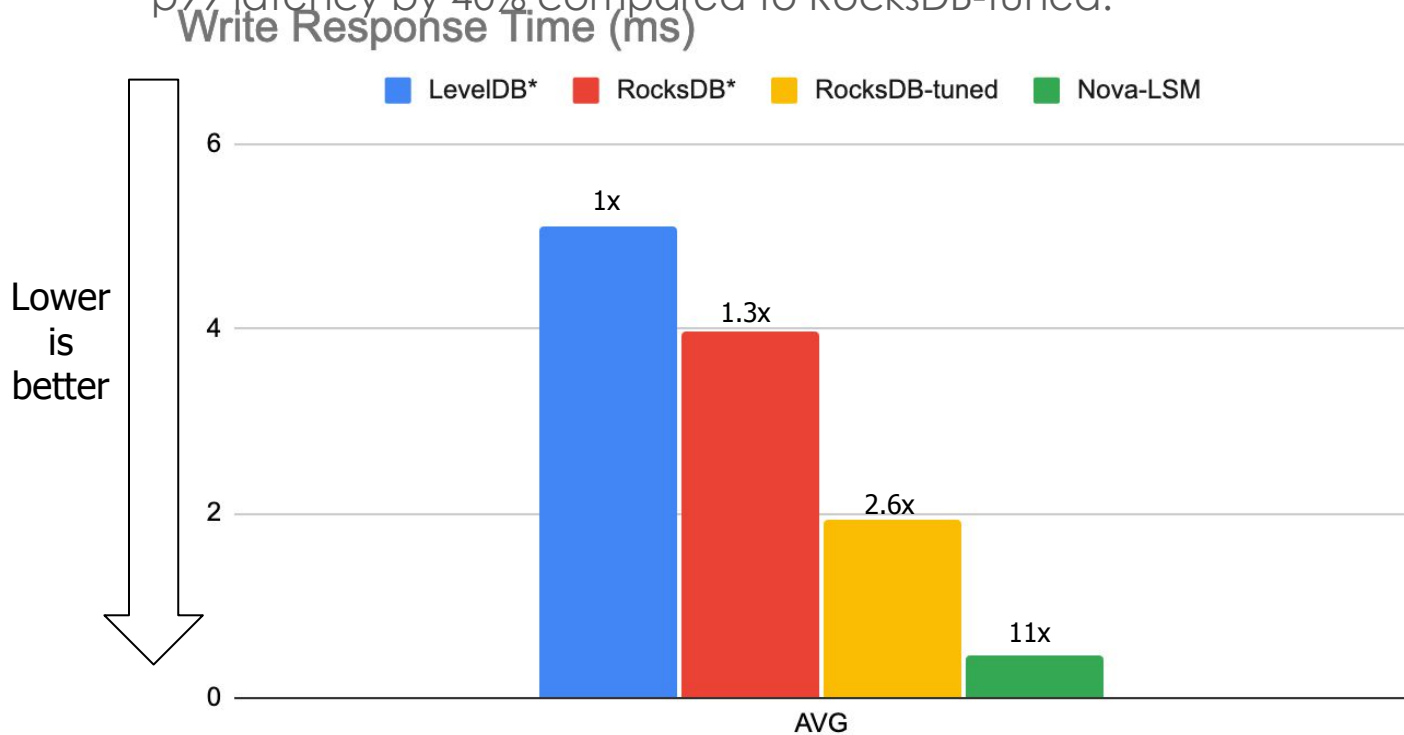
Evaluation: Throughput

- Nova-LSM provides 9x to 18x higher throughput.



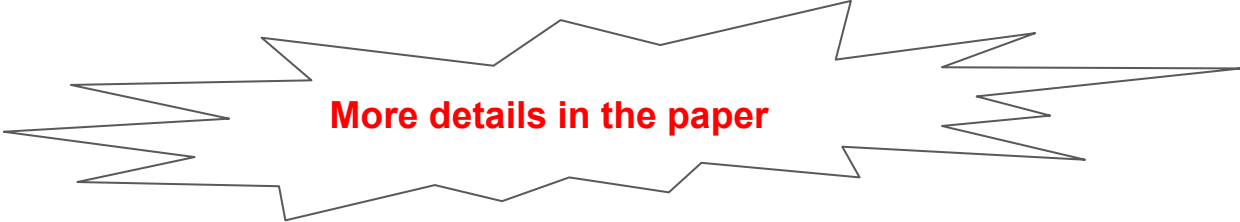
Evaluation: Response Time

- Nova-LSM improves:
 - Average response time by 11x.
 - p99 latency by 40% compared to RocksDB-tuned.



Nova-LSM

- Scale components independently for different workloads.
- Use lookup index and range index to fasten reads.
- Scatter blocks of a SSTable across disks using power-of-d.
- Replicate log records via RDMA.
- Recover GBs of memtables under a few seconds.



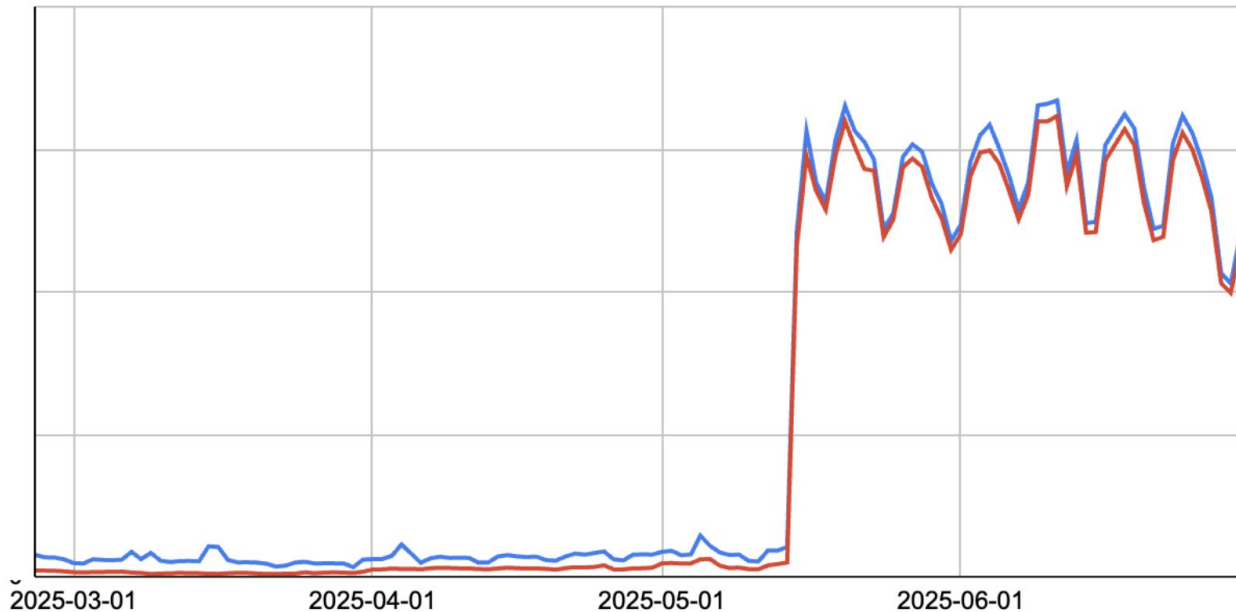
More details in the paper

Lakebase

Lakebase

Agent creates many more databases than human.

— Total new DB count — Agent created db count



Agent



...



Make me an fitness app that finds pet-friendly places

✓ Checkpoint made 4 months ago • Free

Started building your app

🔄 Rollback here

<> Changes

I want to navigate my fitness history

✓ Checkpoint made 4 months ago

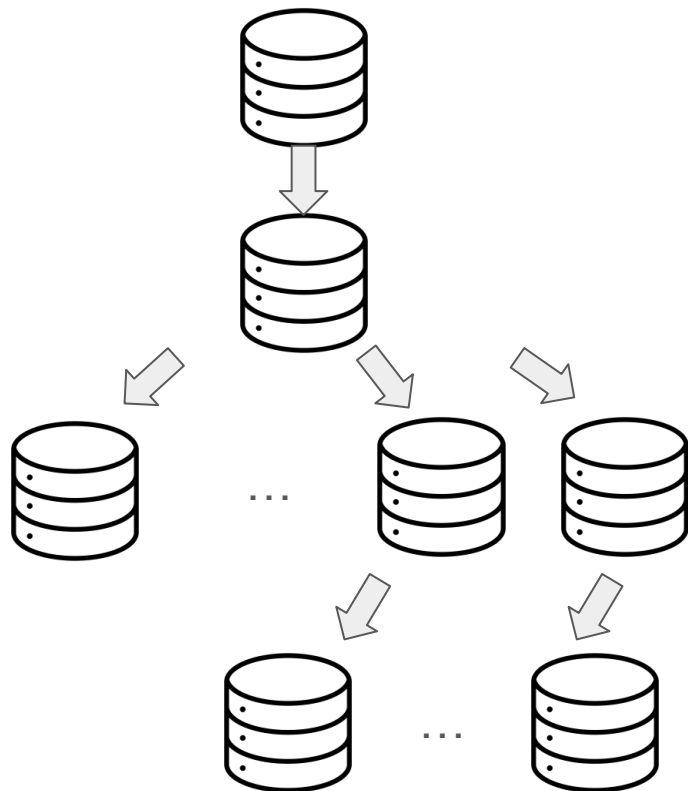
Build core user interface and navigation for the fitness app

🔄 Rollback here

<> Changes

🕒 Preview expired

Agent



Make me an fitness app that finds pet-friendly places

✓ Checkpoint made 4 months ago • Free

Started building your app

🕒 Rollback here <> Changes

I want to navigate my fitness history

✓ Checkpoint made 4 months ago

Build core user interface and navigation for the fitness app

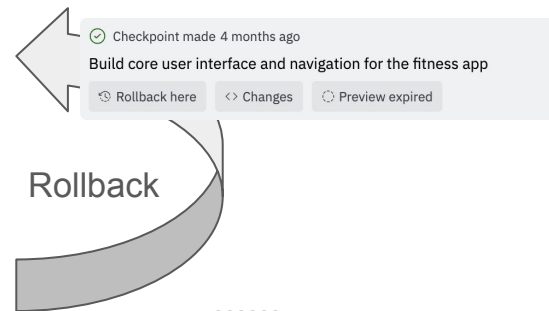
🕒 Rollback here <> Changes ⌚ Preview expired

.....

✓ Checkpoint made 4 months ago

Build core user interface and navigation for the fitness app

🕒 Rollback here <> Changes ⌚ Preview expired

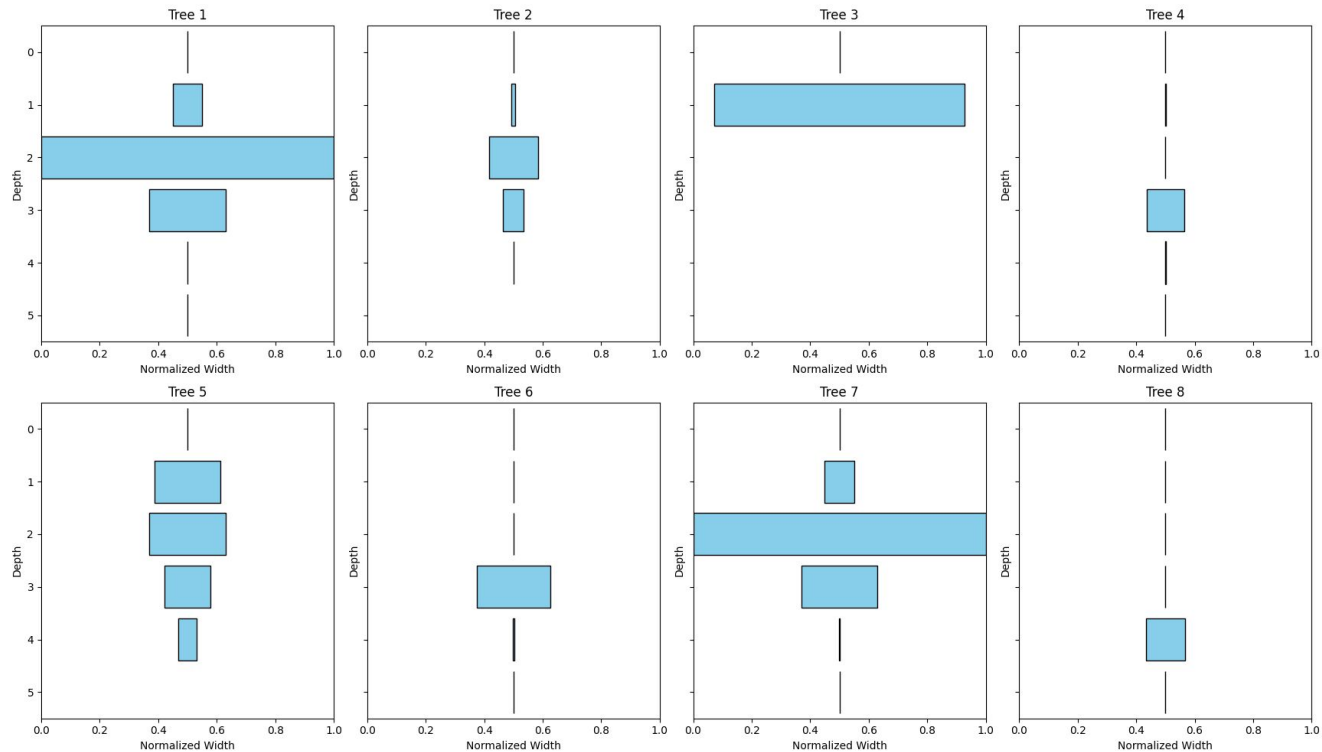


✓ Checkpoint made 4 months ago

Build core user interface and navigation for the fitness app

🕒 Rollback here <> Changes ⌚ Preview expired

Interesting Tree Shapes



Observations

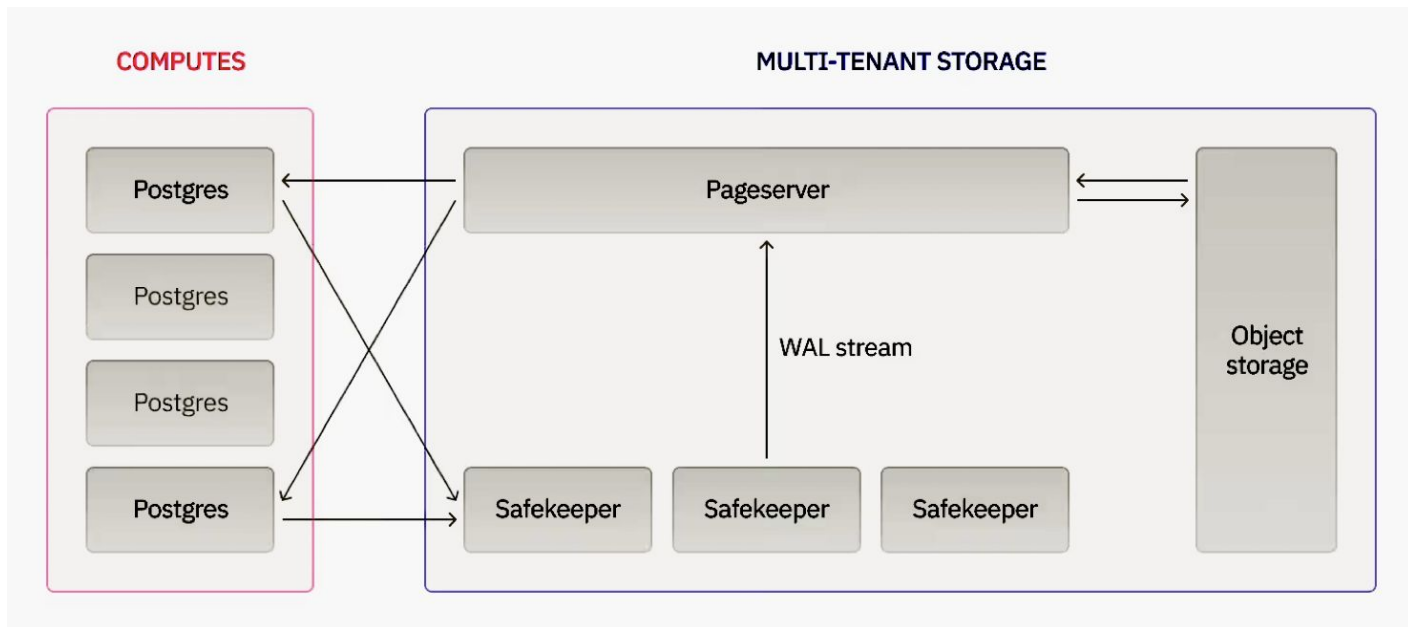
- Database startup time is on the critical path for every prompt.
- Need fast copy and restore on storage.
- Short-lived computes.
- Diverse tree shapes.
- Final artifact is a production ready database.

Lakebase

- Super fast database creation.
- Virtually free branching. => also changes developer workflows.
- Storage and compute separation.
- 100% Postgres compatible.

Architecture

Object storage is the database.



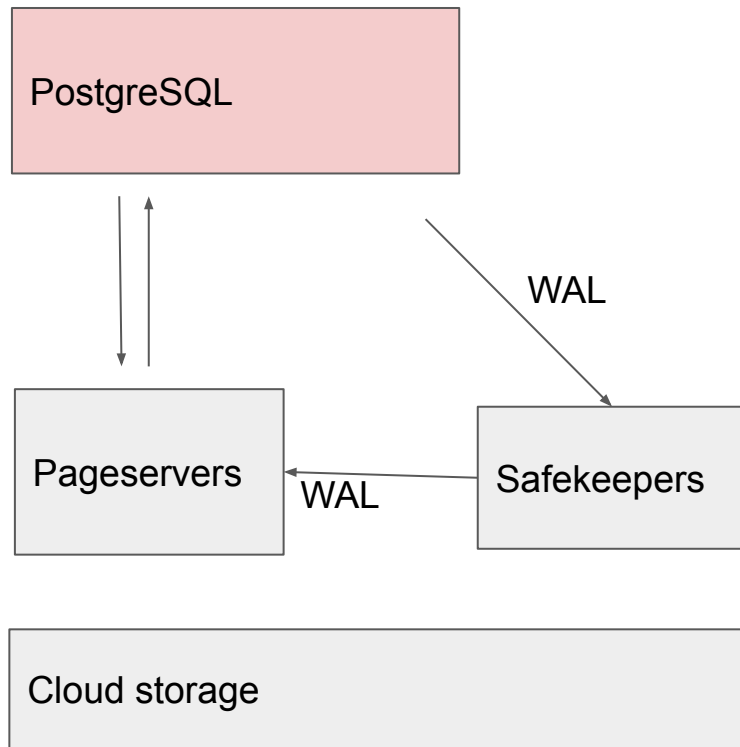
Compute Storage Separation

PostgreSQL:

- Stateless
- Write WAL records to safekeepers
- Read pages from page servers.
- Pages are cached locally.

* WAL = write-ahead log. Postgres WAL is a byte stream.

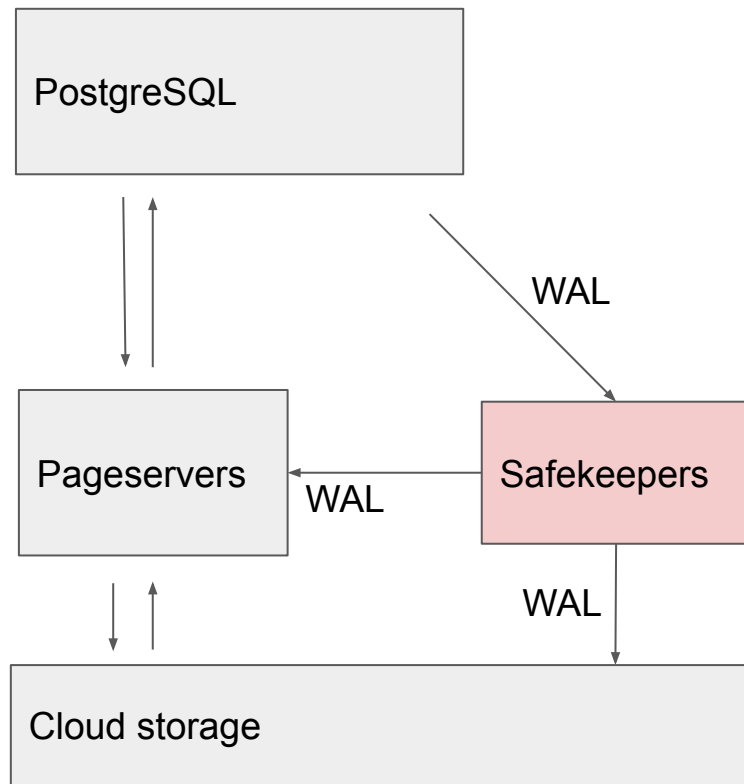
* PG write wal before data.



Write path

PostgreSQL streams the WAL to the safekeepers

- Three running safekeepers
- Consensus algorithm based on Paxos
- Ensures durability of recent transactions
- WALs are backed up to cloud storage.



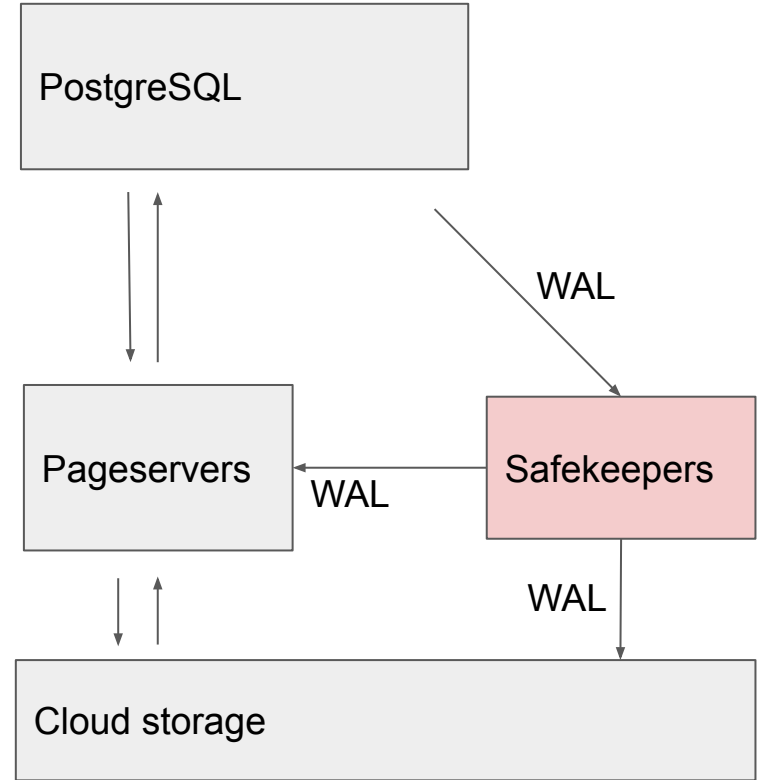
Write path

Fast commits:

- Writes to SK Quorum => write is durable.

Durability:

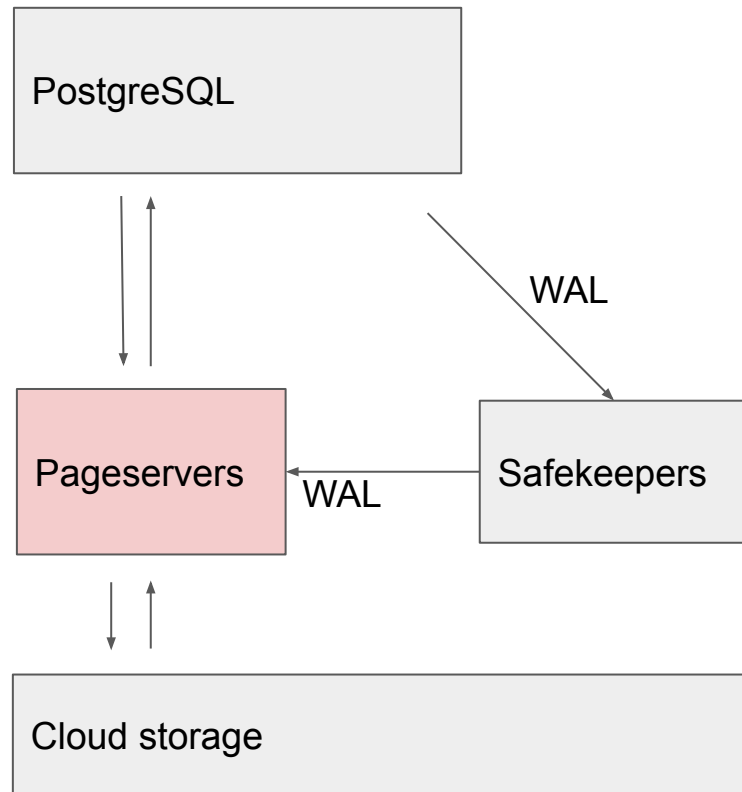
- Cloud storage is 99.999999999% durable.



Write path: Page Servers

Page server caches materialized pages and also wal records.

- Digests the PostgreSQL WAL
- Writes them into immutable files.
- Uploads files to cloud storage
- Local SSDs for caching

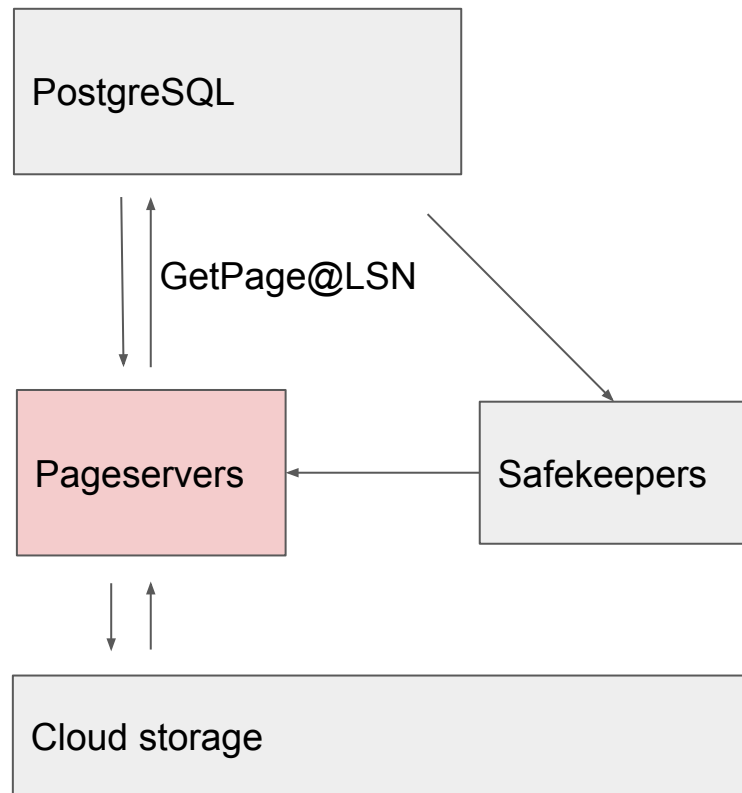


Read path: Page Servers

GetPage@LSN:

- Read the page X @ LSN Y.

Postgres WAL is a byte stream. LSN is the offset into the stream.



Fast compute startup

- Postgres compute only needs metadata to start.
 - All data are in remote storage and can be fetched on demand.
- No need for crash recovery.
- CP maintains a pools of compute resources.

Auto Scaling

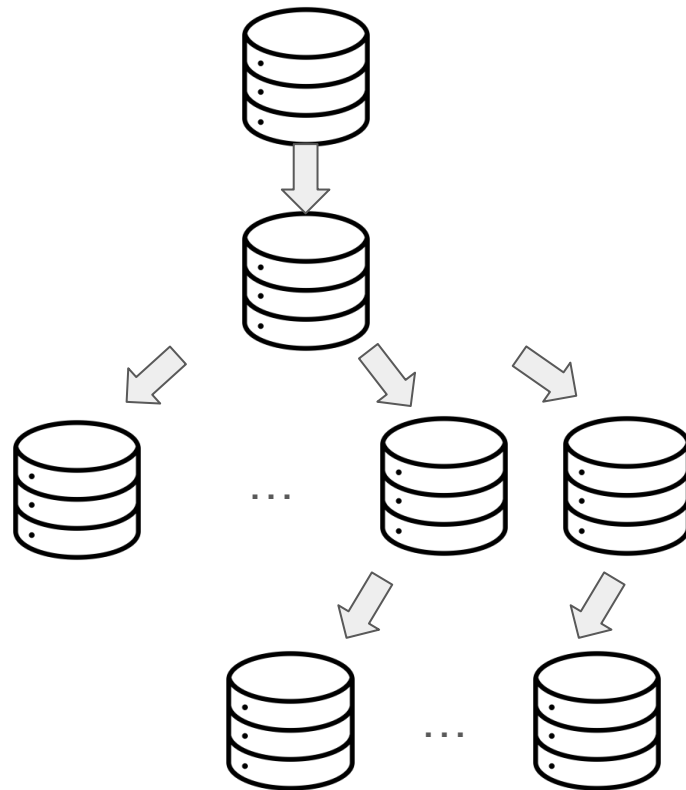
- Compute monitors the working set size.
- It auto scales the compute (cache size & CPU)



Branching

Traditional way to clone a database requires physical data copy.

- The time spent is on the order of magnitude of the database size.
- A multi-TB database may take days to copy.
- A user prompt cannot wait for days to respond.

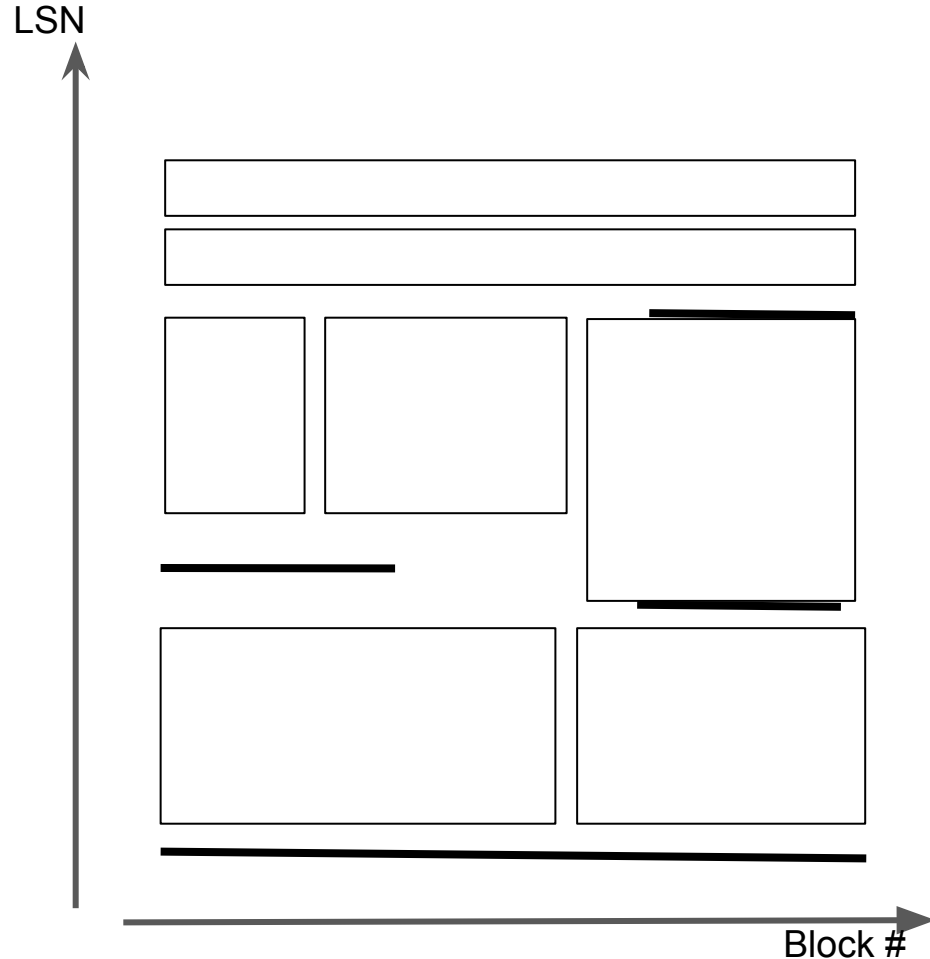


Branching

- Zero-copy clone. => instantaneous branching.
- GetPage on a child branch may recurse to its parents.
- Write on the parent/child branches continues as is.

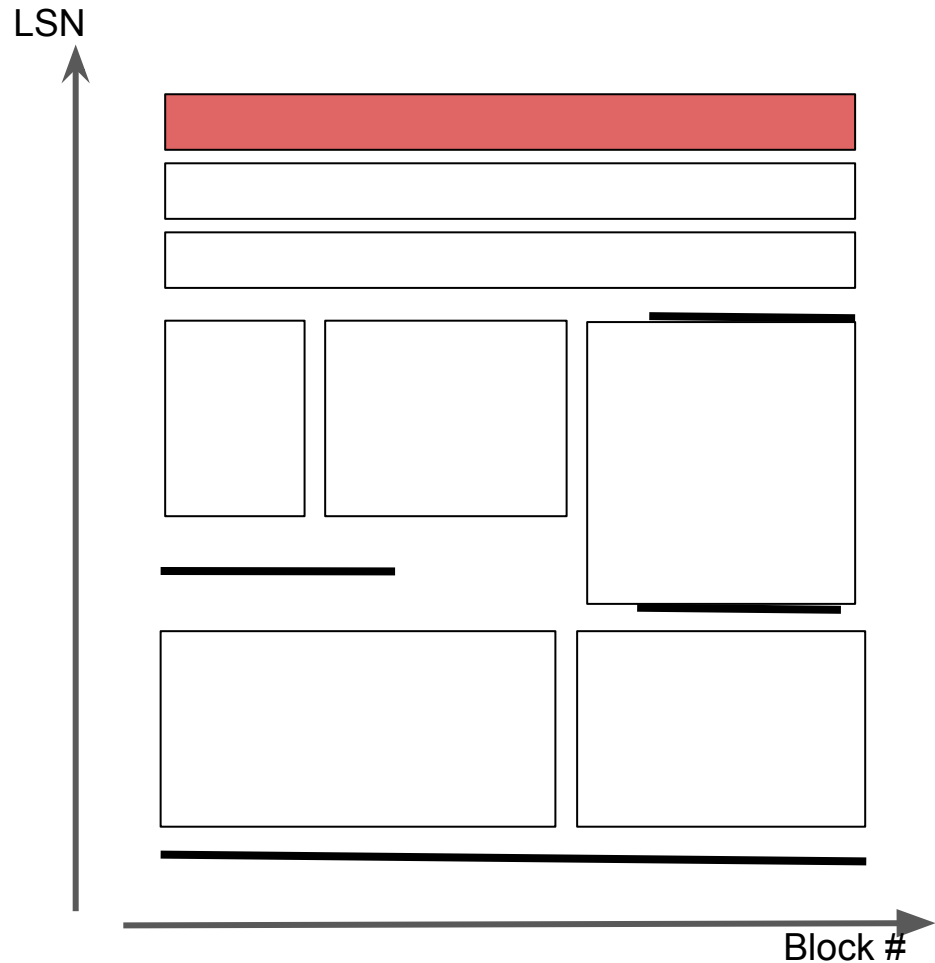
Storage format

- 2D format
- X-axis is block ID
- Y-axis is LSN
- Rectangles are delta layers
- Horizontal bars are image layers
- Each file is roughly same size



Processing incoming WAL

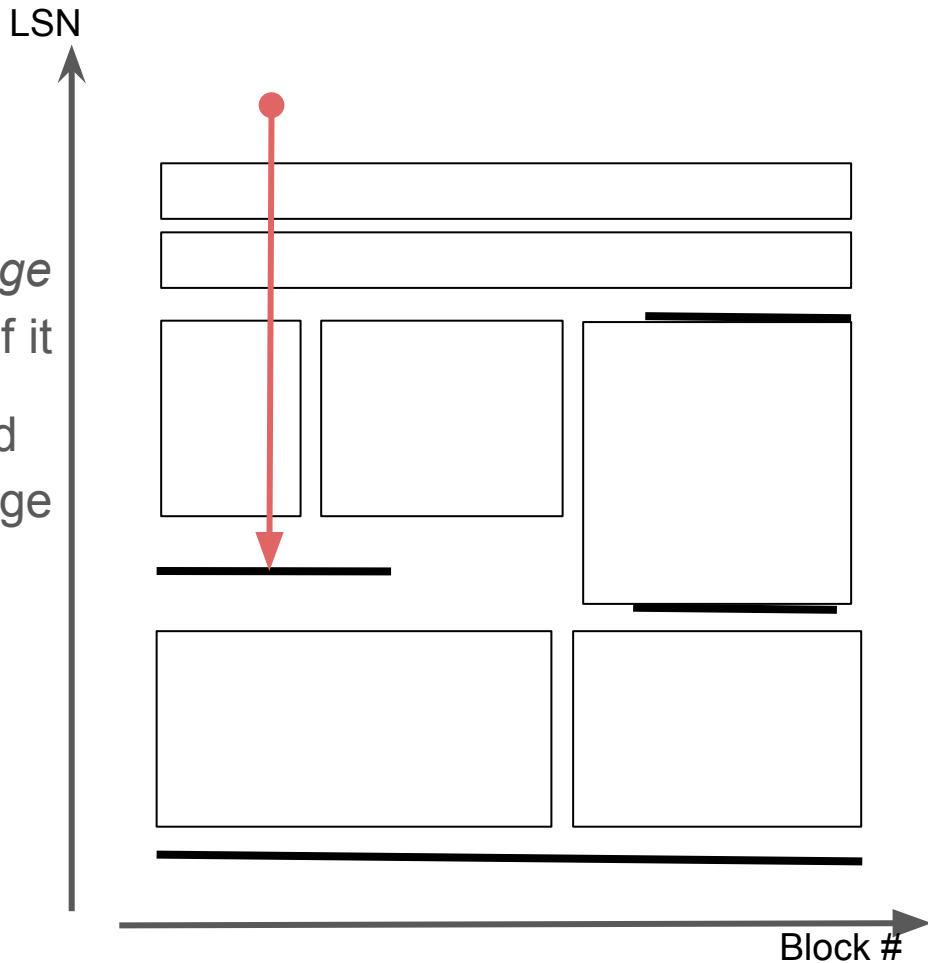
New delta layers are added to the top



GetPage

To reconstruct a page version, `GetPage@LSN` needs to find the last *image* of the page, and all WAL records on top of it

- Search starts at the given block # and LSN, visit layers until you find an image
- Delta layers may contain images
- Search stops at image layer



GetPage on Child

1. Get page on the current database.
2. While missing pages or no parents:
 - a. `current = current.parent`
 - b. `current.GetPage()`

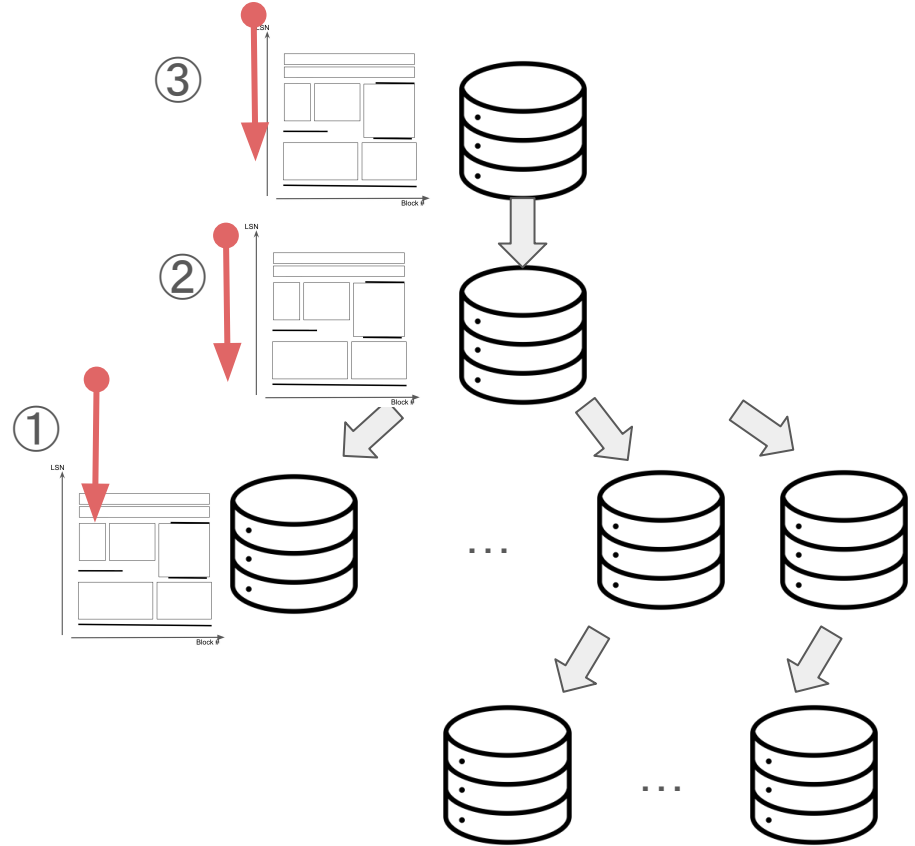
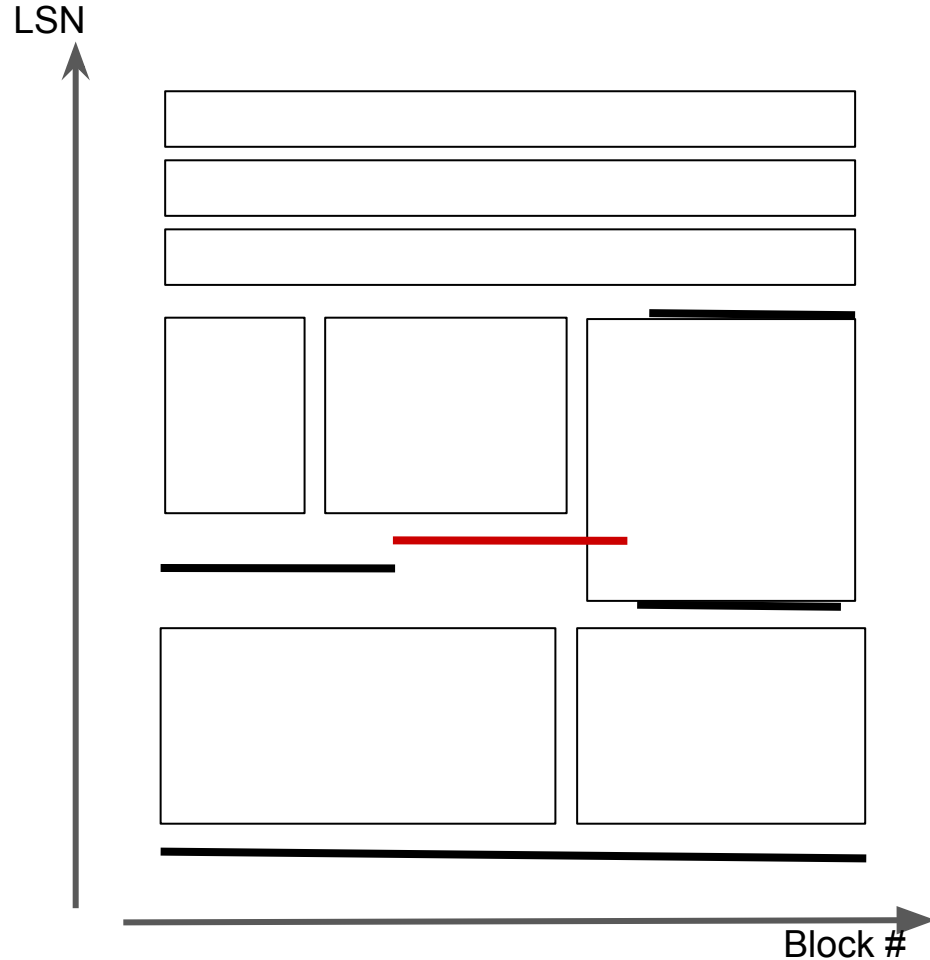


Image layer creation

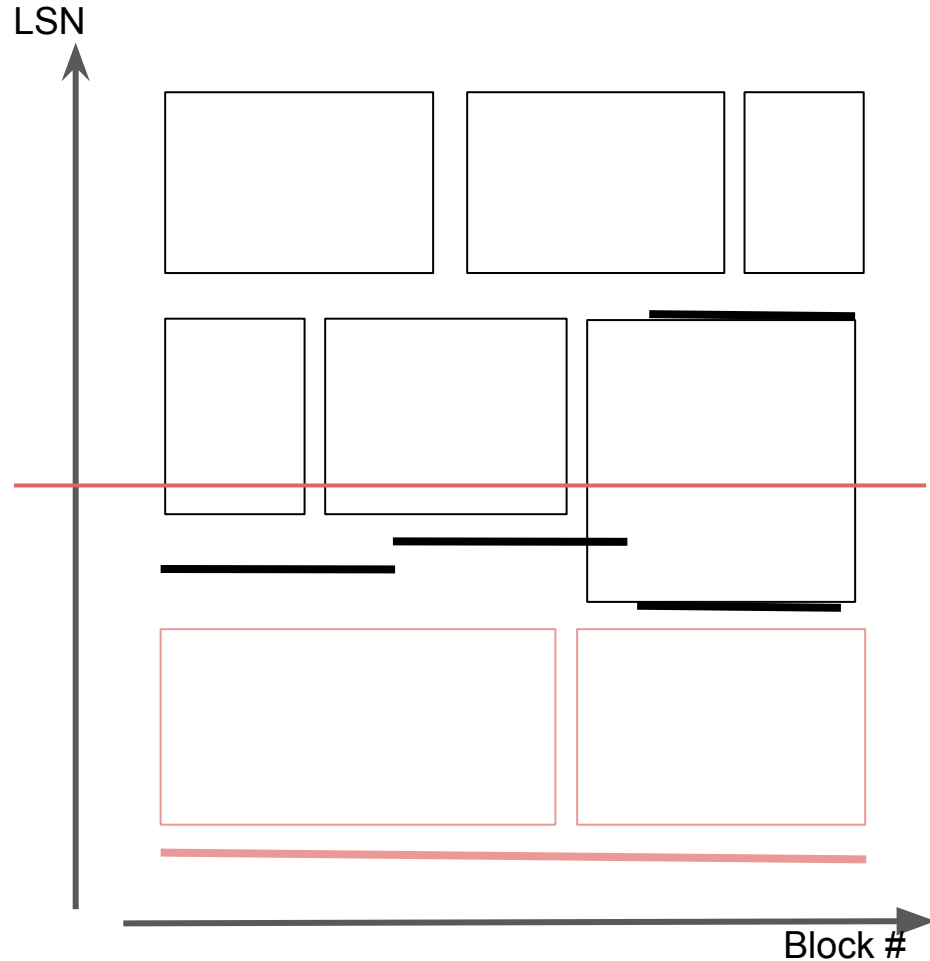
Image layers are created in the background:

- To speed up searches
- To allow garbage collection



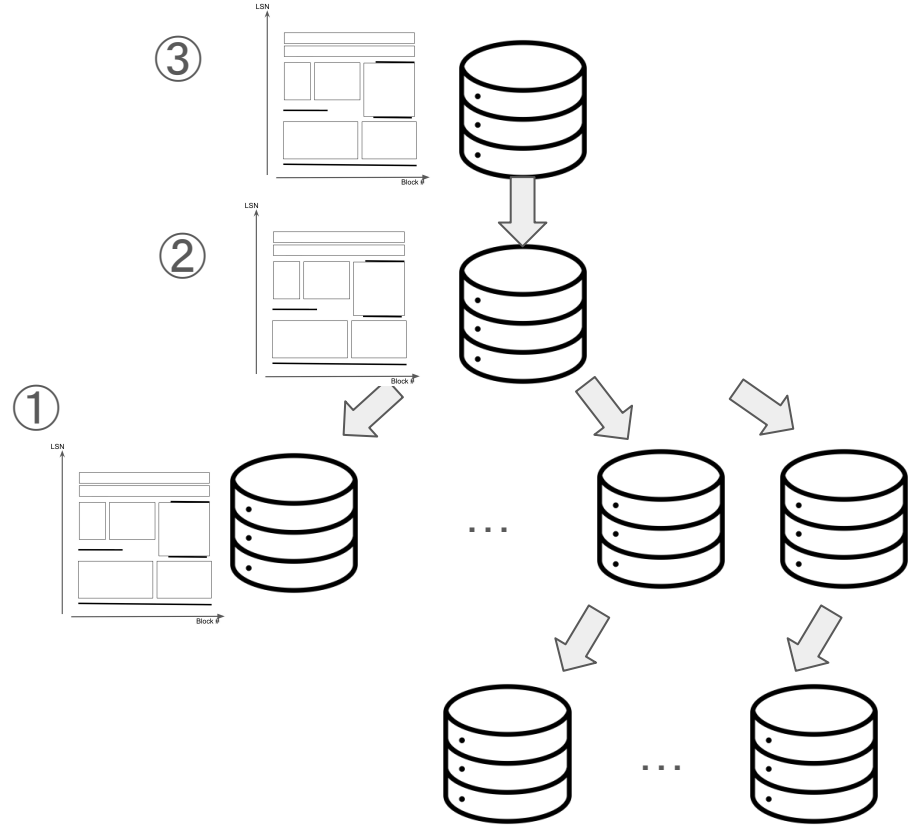
Garbage collection

Removes old layer files that are not needed anymore

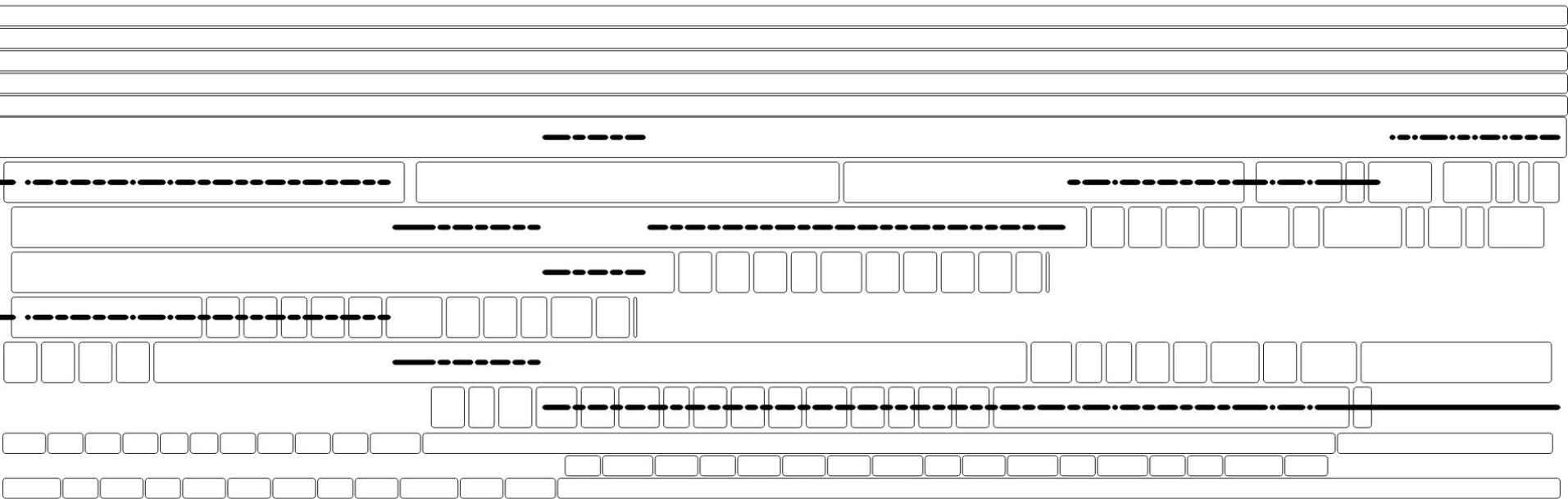


Garbage collection on Child

A parent can only delete a layer file if it's fully covered in every child.



Visualization from a real database



Databricks is Hiring

[Software Engineer - New Grad \(2026 Start\)](#)